VILNIAUS UNIVERSITETAS

MATEMATIKOS IR INFORMATIKOS FAKULTETAS

INFORMACINIŲ SISTEMŲ INŽINERIJOS STUDIJŲ PROGRAMA

Profesinės praktikos ataskaita

Zonos su statinėmis kliūtimis padengimo kelio planavimas.

Justas Remeikis

Praktikos institucija : Vilniaus Universitetas Duomenų mokslo ir skaitmeninių technologijų institutas

Praktikos vadovas : Prof. dr. Virginijus Marcinkevičius

Vilnius

2026

Turinys

Dokumente naudojamos sąvokos ir santrumpos **3**

1. Įvadas **5**
1.1. Tikslas 5
1.2. Uždaviniai 5

2. Analitinė dalis **6**
2.1. Bustrofedono dekompozicija 7
2.2. ZPKP algoritmų apibendrinimas 9
2.3. Sukurti sprendimai ZPKP tyrimų srityje 11

3. Metodinė dalis **13**
3.1. Sistemos paskirtis 13
3.2. Keliami reikalavimai sistemai 13
3.3. Sistemos naudotojai 13
3.4. Funkciniai reikalavimai 14
3.5. Bustrofedono dekompozicijos algoritmas 15
3.6. Internetinės sąsajos architektūra 18
3.7. Įrankių rinkinys sistemos kūrimui 18
3.8. Programavimo kalba sistemos kūrimui 18
3.9. Naudotojo sąsajos karkasas 19
3.10. Naudotojo sąsajos duomenų bazė 19
3.11. Fizinė duomenų bazės struktūros schema 20
3.12. Internetinės naudotojo sąsajos ir algoritmo sujungimas 26
3.13. Sistemos paleidimas ir naudojimas 26

4. Įgyvendinimo dalis **27**
4.1. Zonos padengimo kelio planavimo simuliacinės aplinkos įrankio realizavimas 27
4.2. Zonos padengimo kelio planavimo algoritmo realizavimas 35
4.3. Zonos padengimo kelio planavimo algoritmo testavimas 39
4.4. Skaičiavimo pajėgumų testavimas 39

5. Rezultatai ir apibendrinimas **40**

1 priedas. Sukurtos sistemos programinis kodas **43**

2 priedas. Dirbtinio intelekto įrankių naudojimo deklaracija **44**

Dokumente naudojamos sąvokos ir santrumpos

Terminas ar santrumpa	Aprašymas
CPP	Zonos padengimo kelio planavimas (angl. Coverage Path Planning).
ZPKP	Zonos padengimo kelio planavimas.
BCD	Bustrofedono ląstelinė dekompozicija (angl. Boustrophedon Cellular Decomposition).
TWPS	Dvikryptė artumo paieška (angl. Two-Way Proximity Search).
Re — DQN	Sustiprintas gilusis Q-tinklas (angl. Reinforced Deep Q-Network).
ROS	Robotų operacinė sistema (angl. Robot Operating System).
API	Programų sąsaja (angl. Application Programming Interface).
JSON	Duomenų formatas (angl. JavaScript Object Notation).
CSV	Kableliais atskirtų reikšmių formatas (angl. Comma-Separated Values).
BFS	Paieška į plotį (angl. Breadth-First Search).
CLOC	Kodo eilučių skaičius (angl. Count Lines of Code).
DFS	Paieška į gylį (angl. Depth-First Search).
PWA	Progresyvi žiniatinklio programa (angl. Progressive Web Application).
SPA	Vieno puslapio programa (angl. Single Page Application).
Ląstelė	Nesikertanti posritis, į kurią skaidoma padengimo zona.
Ląstelinė dekompozicija	Zonos skaidymas į nesikertančias posritis (ląsteles), kuriose padengimo maršrutą galima lengvai nustatyti.
Gretimumo grafas	Grafas, kurio viršūnės žymi ląsteles, o briaunos — gretimumo ryšius tarp jų (angl. Adjacency graph).
Pjūvis	Vertikali linija, braukiama per aplinką dekompozicijos metu (angl. Slice).
Kritinis taškas	Taškas, kuriame pasikeičia pjūvio jungiamumo būsena (angl. Critical point).
Įvykis	Momento, kai pjūvis kerta daugiakampio viršūnę, žymėjimas (angl. Event).
Bustrofedono kelias	Padengimo maršrutas, atliekamas paprastais judėjimais pirmyn-atgal (angl. Boustrophedon path).
Trapecinė dekompozicija	Dekompozicijos metodas, kuriame zona skaidoma į trapecines ląsteles naudojant vertikalią tiesę (angl. Trapezoidal decomposition).

Bustrofedono dekompozicija	Trapecinės dekompozicijos patobulinimas, sujungiantis ląsteles tarp vidinio ir išorinio įvykių (angl. Boustrophedon decomposition).
Iškiloji dekompozicija	Dekompozicijos metodas, skaidantis zoną pagal vidinių kampų geometriją (angl. Convex decomposition).
Neprisitaikantis algoritmas	Algoritmas, besiremiantis tik stacionaria, iš anksto žinoma informacija (angl. Off-line).
Prisitaikantis algoritmas	Algoritmas, naudojantis realaus laiko jutiklių matavimus (angl. On-line).
Godžioji strategija	Optimizavimo strategija, kiekvienu žingsniu pasirenkanti lokaliai geriausią sprendimą (angl. Greedy strategy).

1. Įvadas

Šioje ataskaitoje aprašomi profesinės praktikos, atliktos Vilniaus universiteto Duomenų mokslo ir skaitmeninių technologijų institute, iškelti tikslai, atlikti uždaviniai ir gauti rezultatai.

1.1. Tikslas

Profesinės praktikos tikslas — sukurti internetinę sąsają, skirtą vizualiai simuliuoti zonos padengimo kelio planavimo algoritmų veikimą ir realizuoti zonos padengimo kelio planavimo algoritmą bei jį ištestuoti sukurtoje simuliacinėje aplinkoje. Sukurtas sprendimas turėtų pademonstruoti visą tyrimo procesą, kuris leistų tyrėjams koncentruotis į algoritmų kūrimą, o ne į tai, kaip tuos algoritmus atvaizduoti, analizuoti ir palyginti tarpusavyje.

1.2. Uždaviniai

Praktikos uždaviniai:

1. Zonos padengimo kelio planavimo simuliacinės aplinkos įrankio pirminės versijos užbaigimas.
2. Zonos padengimo kelio planavimo algoritmų analizė ir vieno algoritmo pasirinkimas.
3. Zonos padengimo kelio planavimo algoritmo:
    - Realizavimas.
    - Testavimas simuliacinėje aplinkoje.
    - Pritaikymas „ESP32“ mikrovaldikliui.
4. Skaičiavimo pajėgumų testavimas.
5. Dokumentacijos parengimas.

2. Analitinė dalis

Profesinės praktikos tikslas — sukurti simuliacinę aplinką zonos padengimo kelio planavimo (angl. Coverage Path Planning, CPP) algoritmams tirti. Šie algoritmai nustato roboto maršrutą, leidžiantį vengiant kliūčių padengti visą darbinį plotą. Tai aktualu įvairiose robotikos srityse [19]: nuo patalpų valymo ar žemės ūkio darbų iki konstrukcijų inspektavimo bei paieškos ir gelbėjimo operacijų.

Ankstyvame ZPKP tyrime [2] suformuluoti robotų padengimo užduoties reikalavimai. Nors jie skirti dvimatei aplinkai, tie patys kriterijai galioja ir kitiems scenarijams:

1. Robotas turi visiškai padengti zoną, aplankydamas visus jos taškus.
2. Privaloma vengti maršrutų persidengimo.
3. Operacija turi būti nuosekli ir be pasikartojimų.
4. Būtina išvengti visų kliūčių.
5. Valdymui palengvinti naudojamos paprastos trajektorijos.
6. Siekiama optimalaus maršruto esamomis sąlygomis.

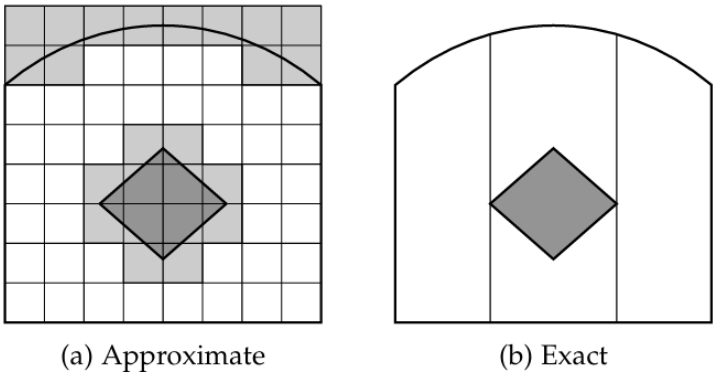
Sudėtingomis sąlygomis ne visada pavyksta įgyvendinti visus reikalavimus, todėl tenka prioritizuoti svarbiausius kriterijus.

ZPKP algoritmai vertinami pagal šiuos kiekybinius rodiklius:

1. Zonos padengimas.
2. Maršruto persidengimas.
3. Posūkių skaičius.
4. Vykdymo laikas.
5. Energijos sąnaudos.

Remiantis [7, 11], siekiant suformuoti optimalią trajektoriją, tikslingiausia minimizuoti posūkių skaičių, nes šie manevrai sunaudoja daug energijos dėl stabdymo ir akceleravimo. Tai leidžia optimizuoti bendrą maršruto ilgį, sutrumpinti užduoties atlikimo trukmę bei sumažinti energijos sąnaudas, kartu užtikrinant pilną darbinės zonos padengimą.

ZPKP algoritmai skirstomi į klasikinius (užtikrinančius visišką padengimą, 1 pav. a dalis) ir euristinius (greitai randančius optimalų sprendimą, 1 pav. b dalis). Pagal Choset [3], jie taip pat skirstomi į neprisitaikančius (angl. Off-line) ir prisitaikančius (angl. On-line). Neprisitaikantys algoritmai veikia žinomoje, statiškoje aplinkoje. Prisitaikantys (jutikliais pagrįsti) algoritmai (angl. sensor based coverage algorithms) išankstinių žinių nenaudoja ir remiasi realaus laiko jutiklių duomenimis.



1 pav. *Dvi pagrindinės ląstelinės dekompozicijos klasės: a) — aproksimuota, b) — tiksli [7].*

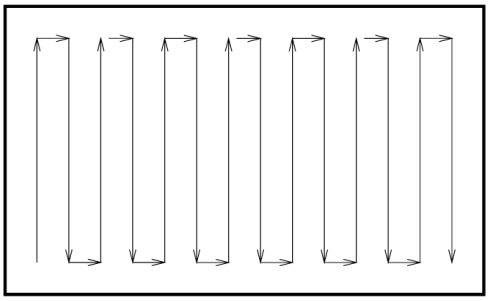
Ankstyvuose tyrimuose [5] maršrutai buvo programuojami rankiniu būdu, nenaudojant automatinių algoritmų. Nors procesą lengvino standartiniai šablonai, jie netiko aplinkoms su kliūtimis.

Kitas paprastas sprendimas — atsitiktinis maršrutas [17]. Ilgainiui taip valomas paviršius tampa švarus, o sistemą įgyvendinti lengva, nes nereikia brangių jutiklių ar skaičiavimo resursų. Visgi didelėse teritorijose laiko ir energijos sąnaudos būtų per didelės, todėl praktikoje dažniau renkamasi sistemingą padengimą užtikrinančius struktūrizuotus metodus.

2.1. Bustrofedono dekompozicija

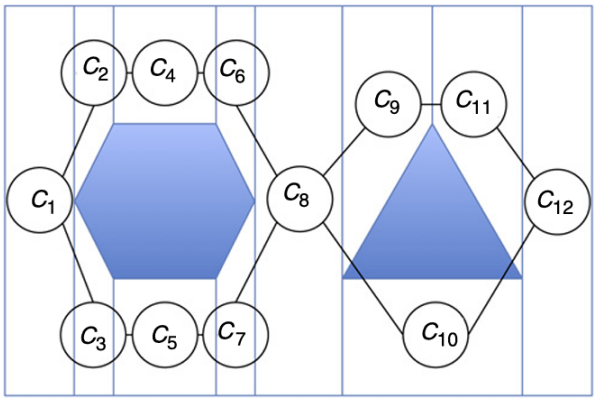
Šiame skyriuje apžvelgiamas pasirinktas ZPKP algoritmas, kurį siekiama įgyvendinti šios praktikos metu. Platesnė ZPKP algoritmų analizė atlikta mokomosios praktikos metu [15].

Bustrofedono dekompozicija (angl. Boustrophedon Cellular Decomposition, BCD) – tai klasikinis metodas, geometriškai skaidantis zoną į nesikertančias ląsteles. Jų visuma užpildo visą plotą, o kiekviena ląstelė padengiama paprastais judesiais, pavyzdžiui, „pirmyn-atgal“ trajektorija (žr. 2 pav.).



2 pav. *Bustrofedono kelias [4].*

Gretimomis vadinamos bendrą ribą turinčios ląstelės. Ląstelinę dekompoziciją atvaizduoja gretimumo grafas, kurio viršūnės žymi ląsteles, o briaunos – jų ryšius (žr. 3 pav.). Tiksli dekompozicija sudaroma per erdvę braukiant tiesę (angl. sweep line): ląstelių ribos formuojamos tiesei susidūrus su įvykiu, pavyzdžiui, kliūties ar zonos viršūne.

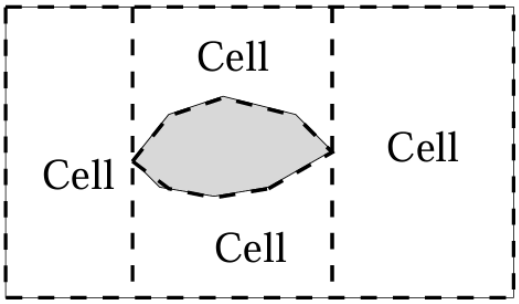


3 pav. *Zonos suskaidymas ląstelėmis ir gretimumo grafo sudarymas naudojant trapecinės dekompozicijos metodą [6].*

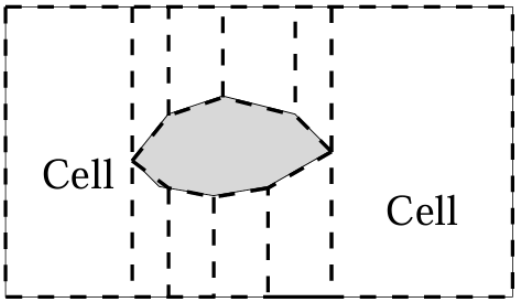
Tikslios sritinės dekompozicijos algoritmai maršrutą generuoja dviem etapais: sritis suskaidoma į ląsteles, sudarant gretimumo grafą, o tada nustatoma visų viršūnių aplankymo seka. Galutinis maršrutas suformuojamas atskirai suplanuojant kelią kiekvienoje ląstelėje ir tarp jų.

Bustrofedono ląstelinė dekompozicija – tai trapecinio metodo patobulinimas, mažinantis perteklinius judesius. Kadangi kliūtys laikomos daugiakampėmis, o aplinka – žinoma, šis metodas laikomas neprisitaikančiu.

Ląstelės tarp vidinio ir išorinio įvykių sujungiamos į vieną. Dėl šio pakeitimo Bustrofedono dekompozicija (žr. 4 pav.) turi mažiau ląstelių nei trapecinė (žr. 5 pav.).

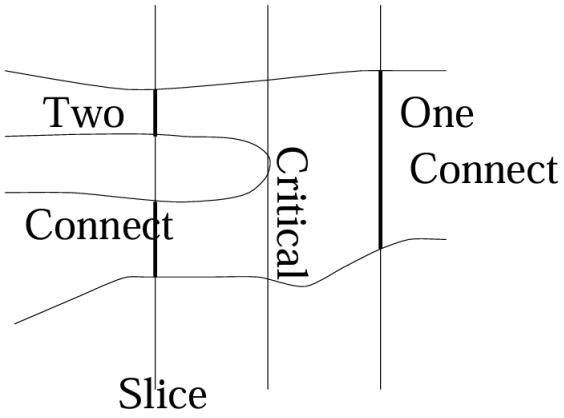


4 pav. *Bustrofedono dekompozicija [4].*



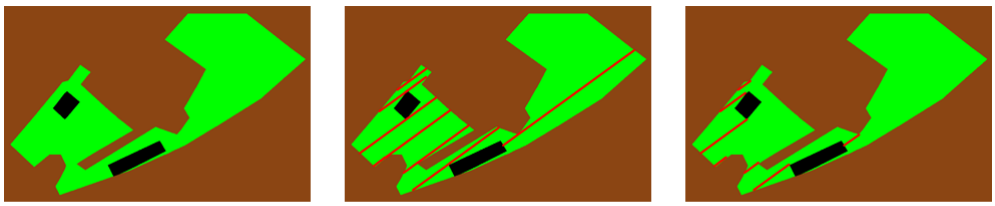
5 pav. *Trapecinė dekompozicija [4].*

Vietoj daugiakampių struktūros šis metodas įvykius nustato pagal pjūvio jungiamumo pokyčius. Tokie įvykiai vadinami kritiniais taškais ir yra plačiai taikomi maršruto planavime. Jų dėka robotas gali padengti zonas kreivose, tolygiose aplinkose (žr. 6 pav.).



6 pav. *Kritiniai taškai yra taškai, kuriuose pasikeičia pjūvio būsena, pvz. iš dviejų pjūvio atkarpų susijungia į vieną [4].*

BCD riboja vienintelė pjūvio kryptis, kuri sudėtingose aplinkose nėra efektyvi visoms ląstelėms. Rezultatams pagerinti taikoma modifikacija [18]: parinkus optimalų pjūvio kampą, minimizuojamas kertamų ląstelių skaičius ir atliekamas jų jungimas (žr. 7 pav.), taip sutrumpinant persidengiantį kelią.



(a) Field shape (b) Decomposed (c) Merged

7 pav. *Optimizuota ląstelinė dekompozicija: (a) zona su kliūtimis, (b) dengiamas plotas padalintas į 12 ląstelių, (c) galutinis rezultatas po ląstelių apjungimo [18].*

Pjūviu pagrįstos dekompozicijos metodai riboti: jie veikia tik tada, kai kiekviena viršūnė pjūvio kryptimi yra unikali. Sudėtingų daugiakampių aplinkoje tai susiaurina galimų krypčių pasirinkimą ir apsunkina optimalios paiešką. Tai būtų galima išspręsti atliekant viršūnių apdorojimą, minimaliai pakeičiant besikertančių viršūnių poziciją, prieš atliekamus metodo skaičiavimus.

2.2. ZPKP algoritmų apibendrinimas

Toliau pateikiamas ZPKP algoritmų palyginimas, gautas atlikus platesnę ZPKP algoritmų analizę mokomosios praktikos metu [15].

Zonos padengimo algoritmai (žr. 2 lentelė) skirstomi į klasikinius ir euristinius. Klasikiniai metodai naudoja ląstelinę dekompoziciją: sritis skaidoma į posritis, o judėjimas planuojamas grafu. Trapecinė dekompozicija kuria ląsteles vertikaliu braukimu, o Bustrofedono metodas jas apjungia, mažindamas perteklinius judesius. Iškiloji dekompozicija zoną į iškiliuosius daugiakampius skaido pagal vidinių kampų geometriją. Prisitaikantys metodai kaip TWPS algoritmas, dengia dinamiškas zonas remdamiesi realaus laiko jutiklių duomenimis be išankstinių žinių. Imtimis pagrįstas C* algoritmas, naudodamas progresinį grafą, darbo metu užpildo padengimo spragas. Re-DQN metodas taiko gilųjį stiprinimo mokymąsi su efektyvesniu tyrinėjimu ir atlygio funkcijomis.

2 lentelė. *ZPKP algoritmų palyginimas*

Algoritmas	Formatas	Tipas	Komentarai	
			Privalumai	**Trūkumai**
Trapecinė dekompozicija	Daugiakampė	Neprisitaikanti	• Paprasta realizacija su vertikaliu braukimu.	• Aplinkose su daugybe viršūnių sukuriama daug ląstelių. • Neveikia, kai daugiau nei 1 viršūnė kerta pjūvio tiesę.
Bustrofedono dekompozicija	Daugiakampė	Neprisitaikanti	• Mažesnis ląstelių skaičius lyginant su trapecine.	• Neveikia, kai daugiau nei 1 viršūnė kerta pjūvio tiesę.
Iškiloji dekompozicija	Daugiakampė	Neprisitaikanti	• Išvengia pjūvio tiesės apribojimų. • Optimalesnės ląstelės lyginant su Bustrofedono dekompozicija.	• Skaičiavimo laikas auga eksponentiškai.
Iškiloji dekompozicija (modifikacija)	Daugiakampė	Neprisitaikanti	• Euristiniai metodai pagreitina skaičiavimus, pasiekiant beveik optimalius rezultatus.	• Kartais praleidžiamos geriausios kombinacijos.
Prisitaikantis (TWPS)	Tinklelis	Prisitaikanti	• Skaičiavimams nebrangus ir efektyvus. • Nesiremia išankstinėmis žiniomis apie aplinką. • Generuoja trumpiausią įmanomą grįžimo kelią.	• Reikia modifikuoti grįžimo algoritmą dinamiškoms kliūtims.
Re — DQN	Tinklelis	Prisitaikanti	• Aukštas prisitaikymo lygis.	• Reikalingi dideli skaičiavimo resursai. • Rizika įstrigti lokaliuose optimumuose.

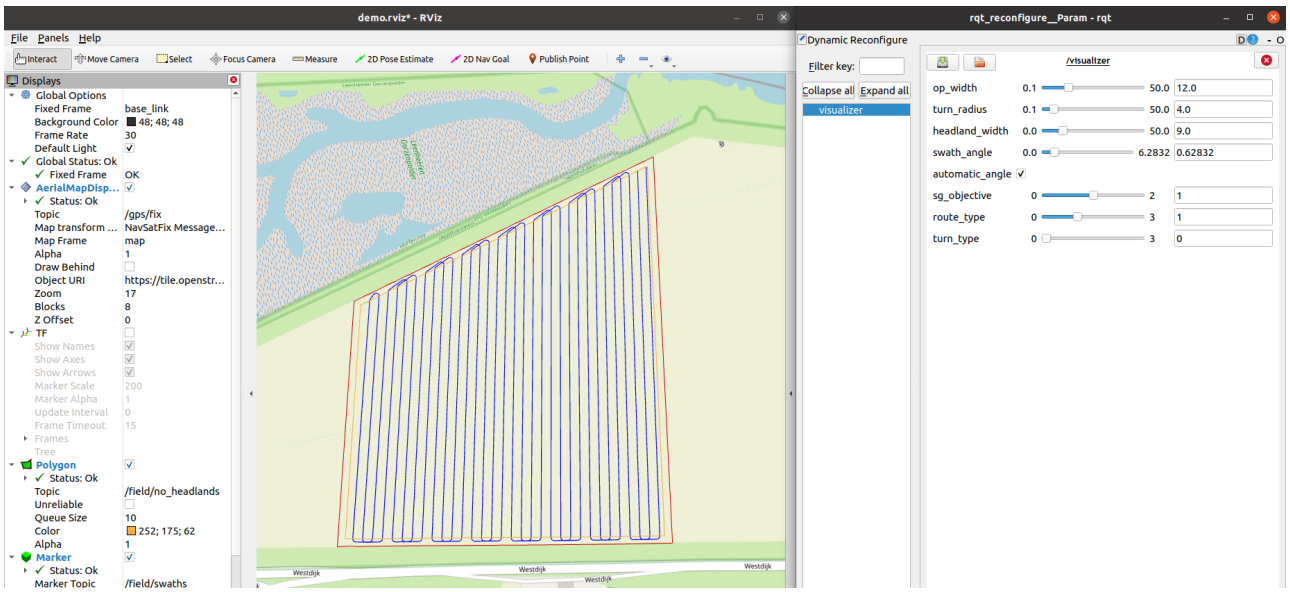
C*	Grafas	Prisitai-kanti	• Skaičiavimams nebrangus ir efektyvus, galima naudoti realiu laiku. • Nesiremia išankstinėmis žiniomis apie aplinką. • Efektyvus ir paprastas padengimas. • Nepalieka padengimo skylių. • Pritaikoma kelių robotų komandai. • Atsižvelgiama į energijos sąnaudų apribojimus.	• Reikalauja išskirtinės aplinkos realizacijos.

2.3. Sukurti sprendimai ZPKP tyrimų srityje

Nors ZPKP sritis tiriama plačiai, trūksta algoritmų įgyvendinimo kodo – autoriai pateikia tik jų aprašymus. Tai lėtina tyrimus, nes lyginamus algoritmus tenka programuoti patiems.

Esami atvirojo kodo sprendimai yra riboti ir sunkiai plečiami. Pažangios, naudotojams patogios programos dažniausiai kuriamos bendradarbiaujant su verslu, todėl jų kodas viešai neprieinamas.

Atvirojo kodo projektas „Fields2Cover“ [14] – viena pažangiausių bibliotekų žemės ūkio technikos padengimo trajektorijoms planuoti. Ji suteikia naudotojo sąsają (žr. 8 pav.) bei karkasą naujų metodų kūrimui ir algoritmų lyginimui. Modulinė, plečiama architektūra pritaikyta įvairioms transporto priemonėms. Pagrindiniai moduliai apima galulaukių, važiavimo juostų, maršruto ir trajektorijos generatorius, taip pat pateikiama sąsaja su ROS sistema.



8 pav. *„Fields2Cover“ naudotojo sąsaja [13]*

Nors įrankis palengvina ZPKP tyrimus, sistemos trūkumai sunkina darbą ir reikalauja papildomo laiko jai perprasti:

- Palaikomos tik statiškos, daugiakampės aplinkos.
- Sistema priklausoma nuo platformos (testuota tik „Ubuntu“), trūksta diegimo instrukcijų kitoms OS.
- Sudėtingas diegimas ir konfigūravimas.
- Naudotojo sąsaja neaprašyta.
- Bibliotekos naudojimas kodo lygmeniu sudėtingas, palaikomos tik „C++“ ir „Python“ kalbos.
- Projektas neaktyvus (paskutiniai pakeitimai prieš 10 mėn.).
- Nemoderni architektūra sunkina tolesnę plėtrą.
- Sistema labiau skirta galutiniam naudojimui, ne ZPKP algoritmų kūrimui ir tyrimams.

Nors ZPKP įrankių yra, tyrimams trukdo jų sudėtingumas: jie nepatogūs naudotojui ir reikalauja gilaus bibliotekų supratimo. Sukurti įrankiai nėra fokusuoti į ZPKP algoritmų kūrimo procesą.

3. Metodinė dalis

3.1. Sistemos paskirtis

Kuriama internetinė ZPKP (zonos padengimo kelio planavimo) simuliacinės įrangos sąsaja skirta tyrėjams ir algoritmų kūrėjams, siekiantiems greitai konfigūruoti ir lyginti skirtingus padengimo algoritmus. Sistema suteikia vieningą darbalaukį aplinkų kūrimui, algoritmų paleidimui bei rezultatų analizei. Sąsaja leidžia interaktyviai modeliuoti realistiškas zonas, įtraukiant statinius ir dinaminius objektus. Surinkti duomenys vizualizuojami, kaupiami ir padengiami kiekybiniais rodikliais.

3.2. Keliami reikalavimai sistemai

Identifikuoti šie reikalavimai sistemai:

- Sistema turi leisti kurti, importuoti, redaguoti ir trinti daugiakampėmis ribomis apibrėžtas zonas bei kliūtis, valdant jų parametrus viename darbalaukyje.
- Sistema turi galėti automatiškai įrašyti aplinkos būseną į duomenų bazę, leisti dirbti skirtingose aplinkose.
- Sistema turi suteikti priemones algoritmo pasirinkimui ir parametrų konfigūravimui bei sinchronizuoti algoritmų ir aplinkų suderinamumą.
- Sistema turi leisti vykdyti ZPKP, perduodant aplinkos ir algoritmo duomenis „JSON“ formatu išorinėms skaičiavimo sistemoms ir grąžintus rezultatus išsaugant duomenų bazėje.
- Sistema turi apskaičiuoti ir saugoti padengimo kiekybinius rodiklius.
- Sistema turi vizualiai pateikti padengimo maršrutus darbalaukyje.
- Sistema turi pateikti paaiškinamuosius pranešimus apie klaidas.
- Sistema turi suteikti darbalaukio navigacijos priemones, kad naudotojai galėtų tiksliai koreguoti ir peržiūrėti aplinkas.

3.3. Sistemos naudotojai

Sistema turi tik vieną naudotoją (žr. 3 lentelę). Kadangi programa kuriama lokaliam naudojimui, autentikacijos funkcionalumas nėra reikalingas. Naudotojas turi pilną duomenų kontrolę, yra pilnai atsakingas už įrankio konfigūravimą ir naudojimą.

3 lentelė. *Sistemos aktoriai*

Pavadinimas	Aprašymas

Naudotojas	Naudotojas yra ZPKP algoritmų tyrėjas, kuris nori ištestuoti ir išanalizuoti savo algoritmą skirtingose aplinkose, palyginti rezultatus su kitais algoritmais. Naudotojas, turi pilnas teises naudoti sistemą. Naudotojas gali: • Naršyti darbalaukyje; • Valdyti ZPKP procesą; • Kontroliuoti, keisti aplinką; • Eksportuoti, importuoti duomenis.

3.4. Funkciniai reikalavimai

Pirminėje sistemos realizavimo versijoje siekiama įgyvendinti 4 lentelėje pateiktus funkcinius reikalavimus. Šių reikalavimų detalus aprašymas pateiktas mokomosios praktikos ataskaitoje [15]. Testavimo funkcionalumo funkciniai reikalavimai praleisti, pirminėje versijoje fokusuojamasi į darbą su viena aplinka ir algoritmu vienu metu. Toliau tobulinant sistemą bus siekiama išvystyti testavimo funkcionalumą, darbui su keliomis aplinkomis ir algoritmais vienu metu.

4 lentelė. *Pirminės sistemos versijos funkciniai reikalavimai*

Eil. Nr.	Reikalavimas
1.	Pasirinkti ZPKP algoritmą.
2.	Keisti ZPKP algoritmo parametrus.
3.	Inicijuoti ZPKP.
4.	Vykdyti ZPKP.
5.	Peržiūrėti ZPKP rezultatus.
6.	Atvaizduoti ZPKP rezultatus.
7.	Įrašyti aplinkos duomenis.
8.	Išsaugoti darbinę aplinką.
9.	Atidaryti išsaugotą aplinką.
10.	Sukurti naują aplinką.
11.	Importuoti aplinką.
12.	Redaguoti aplinkos pavadinimą.
13.	Redaguoti leidimą keisti zonos ir kliūčių tipą.
14.	Nustatyti aplinkos tipą.
15.	Nustatyti aplinkos formatą.
16.	Pasirinkti objektą.
17.	Redaguoti objekto ribas.
18.	Nustatyti objekto tipą.
19.	Sukurti naują objektą.

20.	Ištrinti objektą.
21.	Eksportuoti duomenis.
22.	Valdyti darbalaukio peržiūrą.

3.5. Bustrofedono dekompozicijos algoritmas

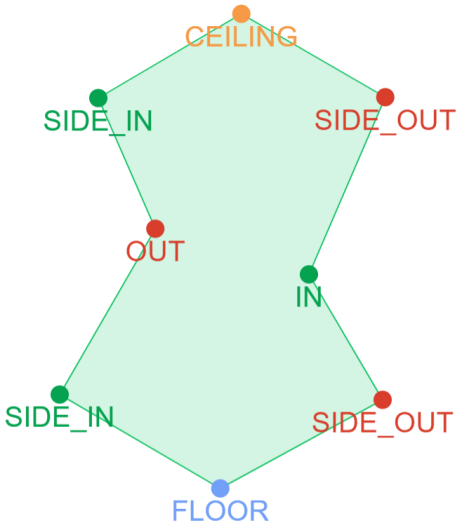
Bustrofedono dekompozicijos algoritmas sudarytas iš 4 skaičiavimo etapų: įvykių generavimo, ląstelių generavimo, ląstelių aplankymo planavimo ir padengimo kelio generavimo (žr. 12 pav.).

Įvykių generavimo etape, kiekvienai viršūnei yra priskiriamas viršūnės tipas, nustatomas pagal viršūnėje susikertančių kraštinių kryptį. Kraštinės kryptis nustatoma pagal kampą tarp kraštinės vektoriaus ir statmenos tiesės braukimo linijai. Išskiriami 6 įvykių tipai:

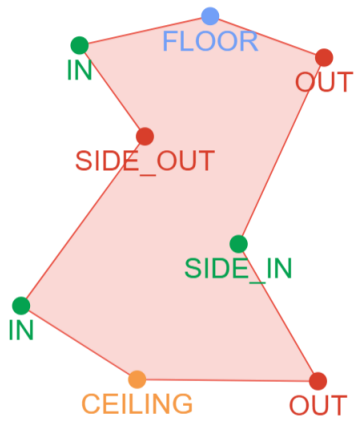
- „IN“ – inicijuojamos 2 naujos ląstelės,
- „SIDE_IN“ – inicijuojama 1 nauja ląstelė,
- „OUT“ – uždaromos 2 senos ląstelės,
- „SIDE_OUT“ – uždaroma 1 sena ląstelė,
- „CEILING“ – tarpinis įvykis tarp „IN“ ir „OUT“,
- „FLOOR“ – tarpinis įvykis tarp „OUT“ ir „IN“.

Zonos ir kliūties „CEILING“ ir „FLOOR“ įvykiai yra apversti (žr. 9, 10 pav.), nes šie įvykiai kuriami iš ląstelės perspektyvos. „CEILING“ naudojama ląstelės viršutinėje kraštinėje, o „FLOOR“ naudojama ląstelės apatinėje kraštinėje.

Įvykių nustatymo taisyklės naudojamos tos pačios zonoms ir viršūnėms, bet zonos viršūnės yra rikiuojamos palei laikrodžio rodyklę, o kliūties viršūnės yra rikiuojamos prieš laikrodžio rodyklę, kas leidžia naudoti tas pačias taisykles skirtingiems objektų tipams.

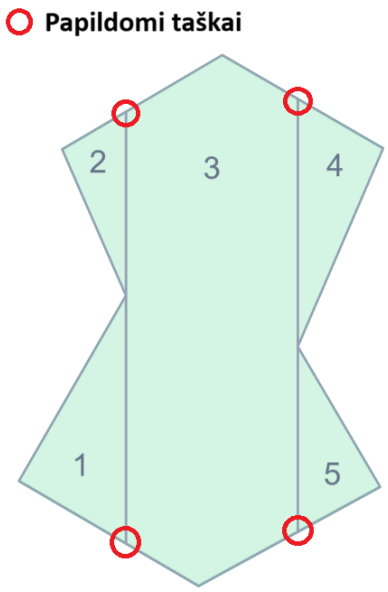


9 pav. *Zonos viršūnių įvykiai*



10 pav. *Kliūties viršūnių įvykiai*

Ląstelių generavimas atliekamas einant per sugeneruotus viršūnių įvykius, braukimo tiesės kryptimi. „IN“ ir „OUT“ įvykių atvejais išskiriamos papildomi taškai, sudaromi brėžiant vertikalią liniją ties šiais įvykiais ir randant artimiausias sankirtas su zonos ar kliūčių kraštinėmis. Šios viršūnės naudojamos kaip tarpiniai „CEILING“ ir „FLOOR“ įvykiai.

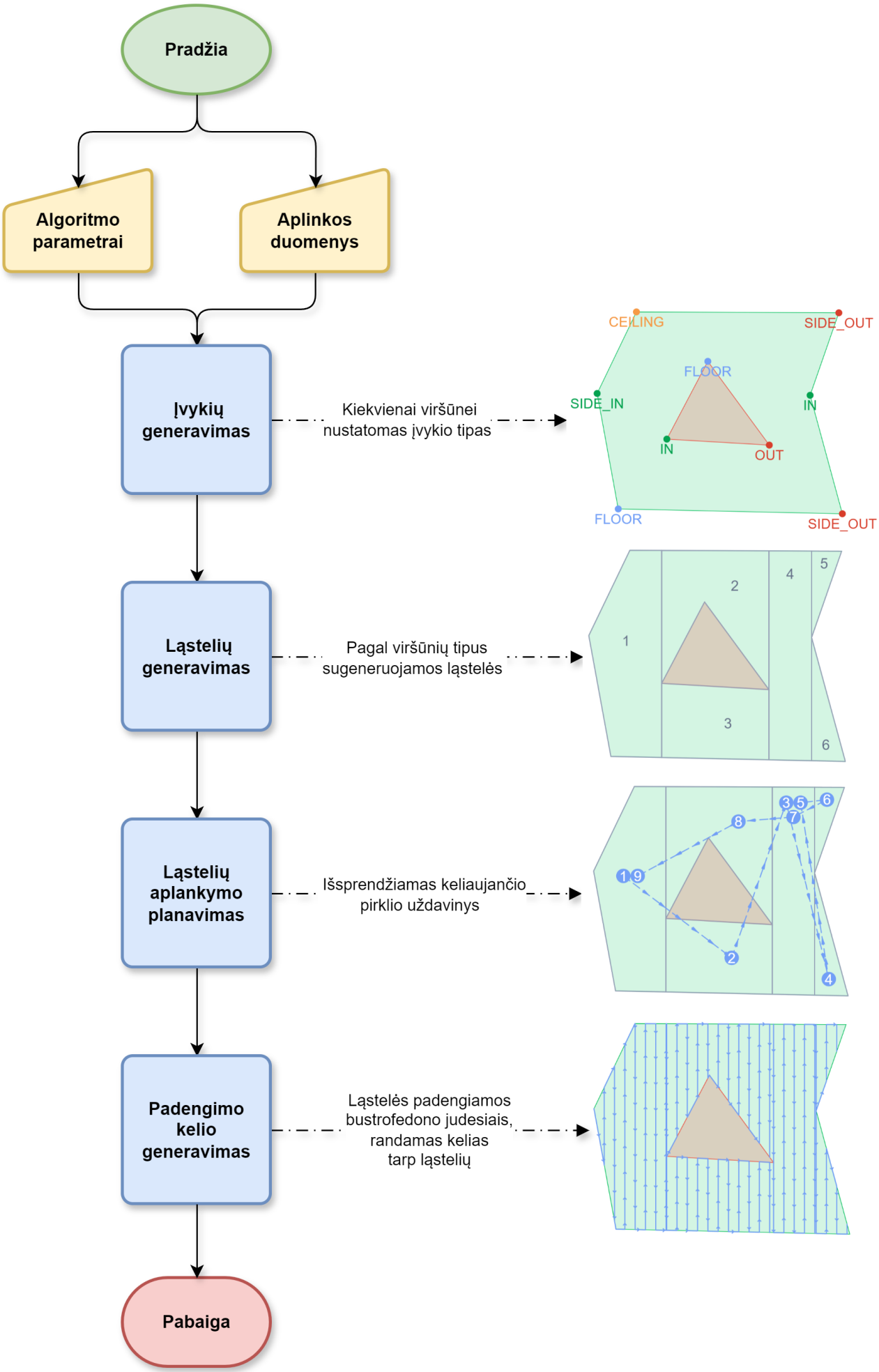


11 pav. *Sugeneruotos zonos ląstelės*

Ląstelių aplankymo planavimas atliekamas sprendžiant keliaujančio pirklio uždavinį, sukuriant ląstelių gretimumo grafą (kuris sudaromas atliekant ląstelių dekompoziciją) ir ieškant sprendimo, kuris aplankytų visas sugeneruotas ląsteles.

Pirmiausia pritaikomas godžiosios paieškos į gylį algoritmas (angl. Depth-First Search, DFS) — pasirenkamas pirmas neaplankytas kaimynas. Kai nebelieka neaplankytų kaimynų, pritaikomas paieškos į plotį algoritmas (angl. Breadth-First Search, BFS), randant artimiausią neaplankytą ląstelę. Pabaigoje, naudojant paiešką į plotį BFS, grįžtama į pradinę ląstelę.

Padengimo kelio planavimas sudarytas iš 2 procedūrų. Pirma procedūra padengia sugeneruotas ląsteles Bustrofedono (pirmyn — atgal) judesiais. Sekanti procedūra, pagal sugeneruotą ląstelių aplankymo planą, sugeneruoja tranzitinį kelią tarp ląstelių.



12 pav. *Bustrofedono dekompozicijos algoritmo diagrama*

3.6. Internetinės sąsajos architektūra

Įrankis kuriamas internetinės sąsajos pagrindu, nes siekiama neapsiriboti viena platforma. Tikslas yra sukurti sistemą, kuri veiktų nepriklausomai nuo naudojamos operacinės sistemos. Naudojant internetinę sąsają galima pasiekti vietinės (angl. native) programos naudotojo patirtį ir naudojant serverį, galima išvengti programos diegimo kiekviename įrenginyje, pasiekiant sistemą per URI adresą. Sistemos naudotojas pagal reikalingą funkcionalumą, galėtų susikonfigūruoti, kad sistema veiktų kaip lokali programa ar kaip SaaS produktas.

Įrankis naudoja vieno puslapio programos (angl. single page app) architektūrą. Visos sistemos funkcijos yra atliekamos viename naršyklės puslapyje, naudojant modalinius langus kaip papildomą funkcinį sluoksnį pranešimams, duomenų įvedimui ir valdymui. Naudojant SPA architektūrą, puslapis niekada nepersikrauna, dinamiškai atnaujinamos tik tos naudotojo sąsajos dalys, kurios pasikeičia. Tokiu būdu pasiekiama vietinės programos naudotojo patirtis naudojantis naršykle.

3.7. Įrankių rinkinys sistemos kūrimui

Sistemos kūrimui tiek internetinės sąsajos, tiek algoritmo pusėje naudojamas „Bun“ įrankių rinkinys, leidžiantis kurti našius ir greitus internetinius serverius (angl. web server). Lyginant su alternatyva — „Node.js“, „Bun“ išsiskiria [9]:

- Didesniu našumu sudėtingų skaičiavimų užduotyse.
- Dvigubai greitesniu HTTP užklausų apdorojimu.
- Didesniu operatyviosios atminties naudojimu.
- Turi integruotus paketų valdymo, testų vykdymo, failų sujungimo (angl. bundler) įrankius.
- Palaiko „Node.js“ paketus.
- Be papildomos konfigūracijos palaiko Typescript programavimo kalbą.
- Integruotu C programavimo kalbos kompiliatoriumi (leidžia patogiai dirbti su C failais).

3.8. Programavimo kalba sistemos kūrimui

Internetinio serverio, naršyklės programos kūrime naudojama Typescript programavimo kalba, vietoje Javascript programavimo kalbos. Typescript leidžia užtikrinti duomenų saugumą (angl. type safety), kurti objektiškai orientuotą kodą, kas pagerina programinio kodo kokybę, patikimumą ir plečiamumą, ypač didesniuose projektuose [8], [1].

Dėl reikalavimo kodą paleisti mikrokontroleriuose, algoritmo realizacijai naudojama C programavimo kalba.

3.9. Naudotojo sąsajos karkasas

Kadangi internetinė naudotojo sąsaja yra interaktyvi ir dinamiška, reikalingas naudotojo sąsajos karkasas, kuris palengvintų realizacijos procesą ir leistų sukurti optimizuotą sprendimą. Pagrindiniai karkasai, bibliotekos naudotojo sąsajos kūrimui yra trys [20]:

- „Angular“ — vienintelis karkasas, turintis integruotus „service worker“ įrankius, kurie svarbūs kuriant vietines programas. Integruota pilna SPA infrastruktūra (maršrutizatorius (angl. router), HTTP klientas, būsenos valdymas, CLI), kas nereikalauja papildomų bibliotekų ieškojimo.
- „Vue“ — našus (mažiausias paketo dydis) karkasas, turintis lengviausią virtualų DOM, kas pagerina našumą dirbant su „canvas“ aplinkomis. Integruotas maršrutizatorius ir reaktyvus būsenos valdymas, naudingas dinamiškose aplinkose.
- „React“ — lankstus, tačiau minimalistinis — atsisakoma integruotų funkcijų mainais į didesnę laisvę. Darbas su „canvas“ yra našus dėl virtualaus DOM skirtumų (angl. diffing), tačiau tiek vietines, tiek maršrutizavimo funkcionalumui reikalingos papildomos bibliotekos.

Dėl plataus bibliotekų pasirinkimo, integracijos su grafinėmis bibliotekomis [10] ir didelio kiekio pavyzdžių bei dokumentacijos, „React“ buvo pasirinkta kaip karkasas internetinės sąsajos kūrimui.

3.10. Naudotojo sąsajos duomenų bazė

Kuriamas įrankis orientuojamas lokaliam naudojimui. Visi sistemos duomenys turi būti išsaugomi naudotojo kompiuteryje be kreipimosi į išorinį serverį. Kuriant internetines programas pagrindiniai kliento pusės (angl. client-side) duomenų išsaugojimo būdai yra [12]:

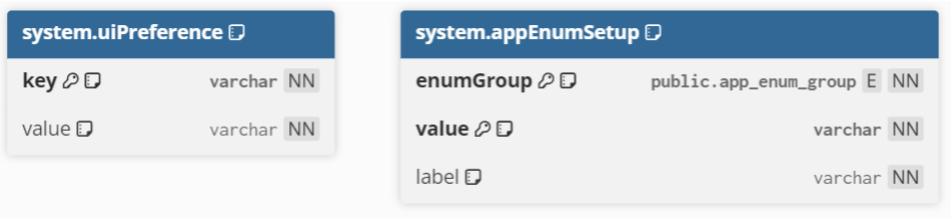
- „LocalStorage” — raktų–reikšmių (angl. key–value) saugykla naršyklėje, skirta nedideliam kiekiui duomenų. Duomenys išlieka uždarius naršyklę ir neturi galiojimo laiko.
- „SessionStorage” — panaši į „LocalStorage“, tačiau duomenys saugomi tik vienos naršyklės sesijos metu.
- „Cookies” — nedidelės apimties duomenų saugojimo mechanizmas, dažniausiai naudojamas sesijoms, autentifikacijai ar nustatymams. Duomenys siunčiami su kiekviena HTTP užklausa į serverį.
- „IndexedDB” — objektinė „NoSQL” tipo duomenų bazė naršyklėje, leidžianti saugoti didesnius struktūrizuotų duomenų kiekius ir atlikti sudėtingesnes užklausas.
- „SQLite” — minimali reliacinė duomenų bazė, dažniausiai naudojama per papildomas technologijas (pvz., „WebAssembly“ ar vietines aplikacijas), leidžianti vykdyti SQL užklausas lokaliai.
- „Cache API” — mechanizmas, skirtas tinklo užklausų (pvz., HTTP atsakymų) talpinimui ir pakartotiniam panaudojimui, dažnai naudojamas kartu su „Service Workers“ kuriant progresyvias žiniatinklio aplikacijas (PWA).

Kadangi kuriamo įrankio specifika yra kaip failo redagavimo priemonė, sudėtingos SQL užklausos duomenų naudojimui nėra vykdomos, todėl naudoti reliacinę duomenų bazę „SQLite” nėra pagrindo. „IndexedDB” duomenų bazė dėl palaikomo indeksavimo, lentelių kūrimo ir paprasto naudojimo be papildomos konfigūracijos, puikiai tinka saugoti aplinkos, algoritmų ir skaičiavimų duomenis. Visi kiti parametrai, tokie kaip atvaizdavimo ar programos būsenos, dėl savo nesudėtingumo gali būti saugomi „LocalStorage” saugykloje.

3.11. Fizinė duomenų bazės struktūros schema

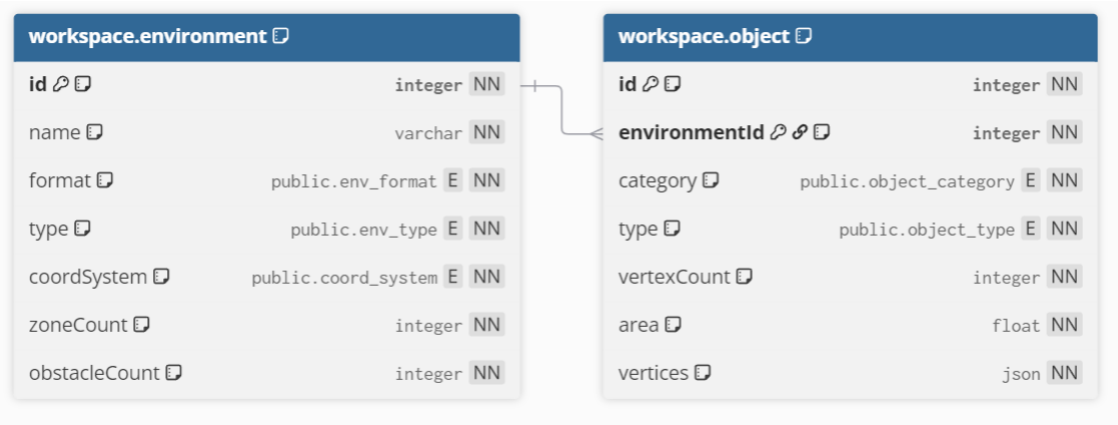
Pasirinkta duomenų bazė „IndexedDB“ leidžia naudoti visus duomenų tipus, kurie gali būti klonuojami. Tai gali būti primityvūs tipai: „integer“, „varchar“, struktūrizuoti tipai: „array“, „json“, media tipai: „ImageBitmap“, tačiau negali būti procesiniai objektai: funkcijos ir „DOM“ objektai. Nors „IndexedDB“ leidžia naudoti įvarius duomenų tipus, tačiau neatlieka duomenų tipo validacijos įvedimo metu. Naudojamus teisingus duomenų tipus turi užtikrinti pati aplikacija, o ne duomenų bazė. Todėl prie duomenų lentelės atributų esantys duomenų tipai nėra duomenų bazės schemos dalis, tai aplikacijos, kuri naudoja duomenų bazę, naudojami duomenų tipai. „IndexedDB“ duomenų bazės schemoje reikalauja aprašyti tik pirminius raktus, papildomi indeksuojami atributai gali būti aprašyti papildomai. Toliau pateikiamos duomenų bazės lentelės atvaizduoja aplikacijos užtikrinamą duomenų struktūrą.

Naudotojo vartotojo sąsajos atvaizdavimo parametrai: valdymo skydelio plotis, stulpelio plotis, informacinio skydelio būsena, išsaugomi „uiPreference“ lentelėje, naudojant paprastą „key“, „value“ struktūrą. Sistemoje naudojamos sisteminės „enum“ vertės laikomos atskiroje „appEnumSetup“ lentelėje (žr. 13 pav.).



13 pav. *Sisteminių duomenų fizinė duomenų bazės schema*

Aplinkos duomenys laikomi „environment“ ir „object“ lentelėse (žr. 14 pav.). „Object“ lentelės esybės priklauso „environment“ lentelės esybei. Kiekviena aplinka gali būti sudaryta iš daugelio objektų, kurie gali būti zonos ar kliūtys.



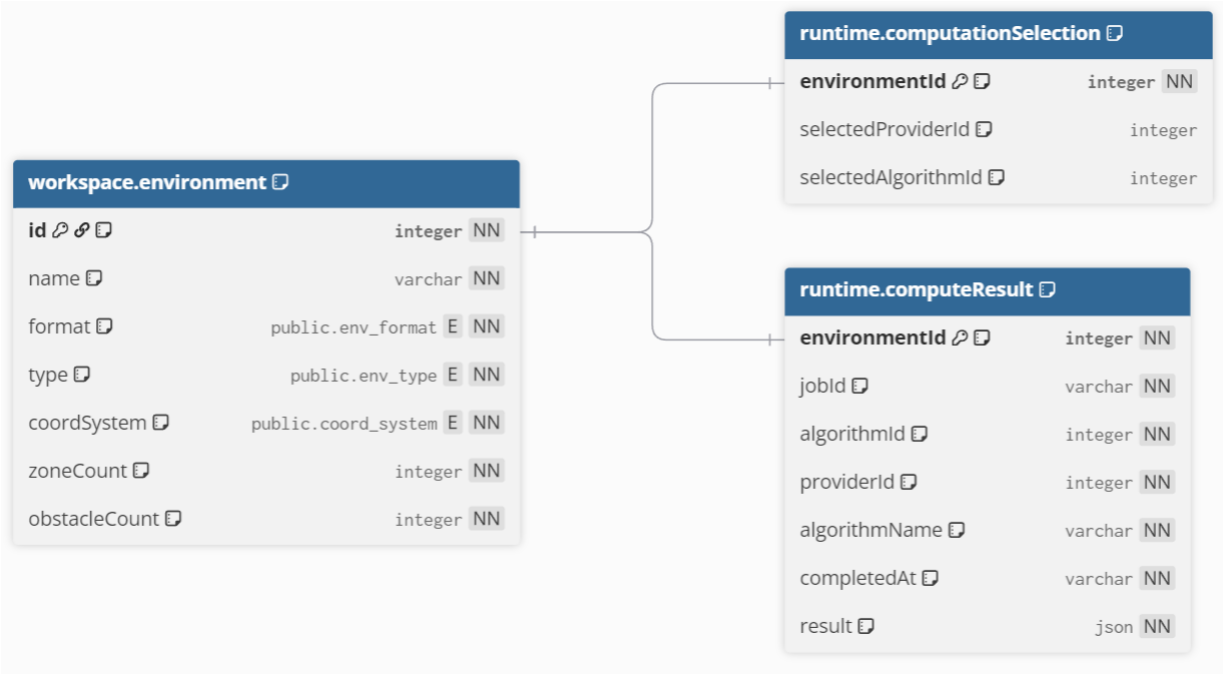
14 pav. *Aplinkos duomenų fizinė duomenų bazės schema*

ZPKP skaičiavimo serveris pasiekiamas per „computationProvider“ lentelėje suvestus duomenis (žr. 15 pav.). Kai naudotojas inicijuoja metaduomenų atnaujinimą, nurodytu „url“ adresu yra išsiunčiama užklausa, kuri grąžintą metaduomenų informaciją įrašo į „computationProviderAlgorithm“, „computationAlgorithmParameterSetup“ ir „algoritmMetricSetup“ lenteles. „ComputationProviderAlgorithm“ lentelėje saugoma informacija apie galimus naudoti algoritmus, kuriuos skaičiavimo serveris įgyvendina. „ComputationAlgorithmParameterSetup“ lentelėje aprašomi algoritmo parametrai, kuriuos naudotojas gali keisti. „AlgoritmMetricSetup“ lentelėje aprašomos algoritmo metrikos, kurias skaičiavimo serveris apskaičiuoja ir internetinė naudotojo sąsaja gali atvaizduoti kaip pavienes reikšmes arba grafikus.

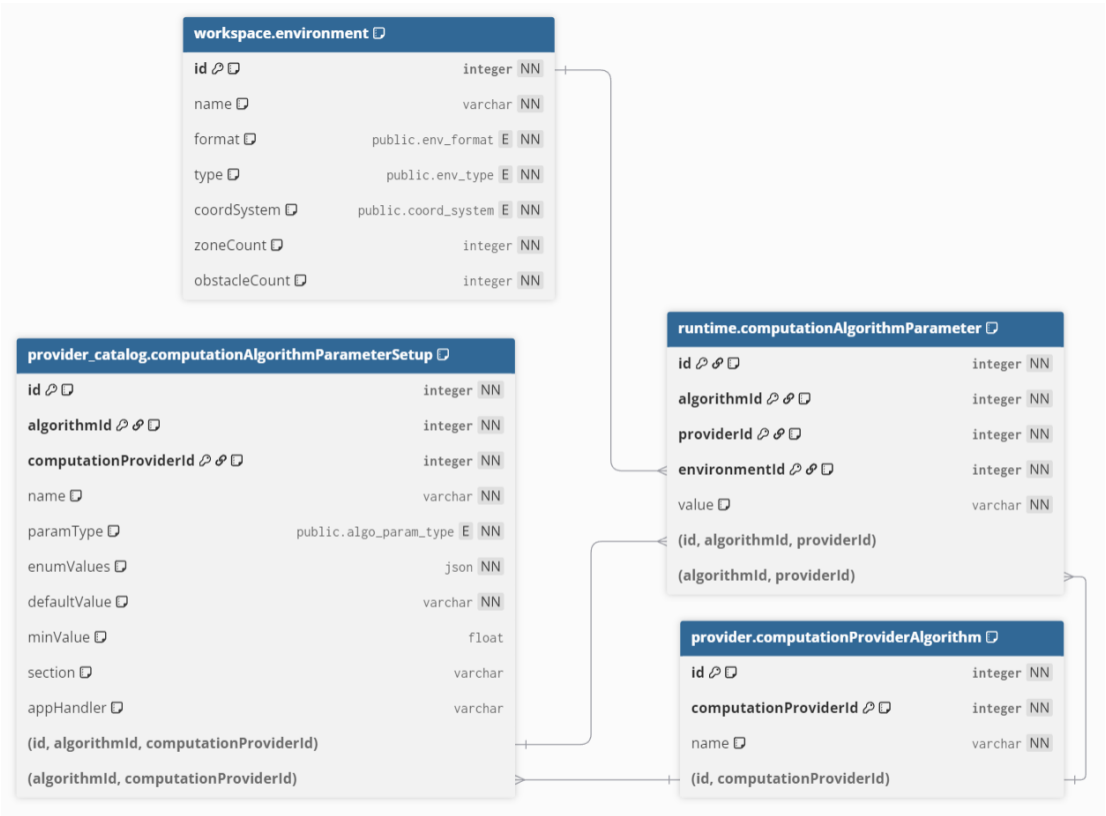
15 pav. *Algoritmo metaduomenų fizinė duomenų bazės schema*

Inicijuojant ZPKP skaičiavimo procesą yra naudojamos pasirinkto algoritmo „computationSelection“ (žr. 16 pav.) ir algoritmo parametrų reikšmės „computationAlgorithmSelection“ (žr. 17 pav.). „ComputationSelection“ yra aplinkos lentelę „environment“ praplečianti lentelė, sukurta atskirti ZPKP procesinius duomenis nuo aplinkos, taip pat „computationSelection“ duomenys nėra naudojami aplinkos eksportavimo metu. „ComputationAlgorithmSelection“ lentelėje saugomi tiktai parametro reikšmės duomenys, parametro duomenys yra saugomi atskirai, „computationAlgorithmParameterSetup“ lentelėje.

ZPKP skaičiavimo serverio atsakymai saugomi „computeResult“ lentelėje, kuri yra dar viena aplinkos lentelę praplečianti lentelė. Lentelėje skaičiavimo rezultatai yra išsaugomi „json“ formatu, kurį apdorojus, rezultatai gali būti atvaizduoti darbo aplinkoje.



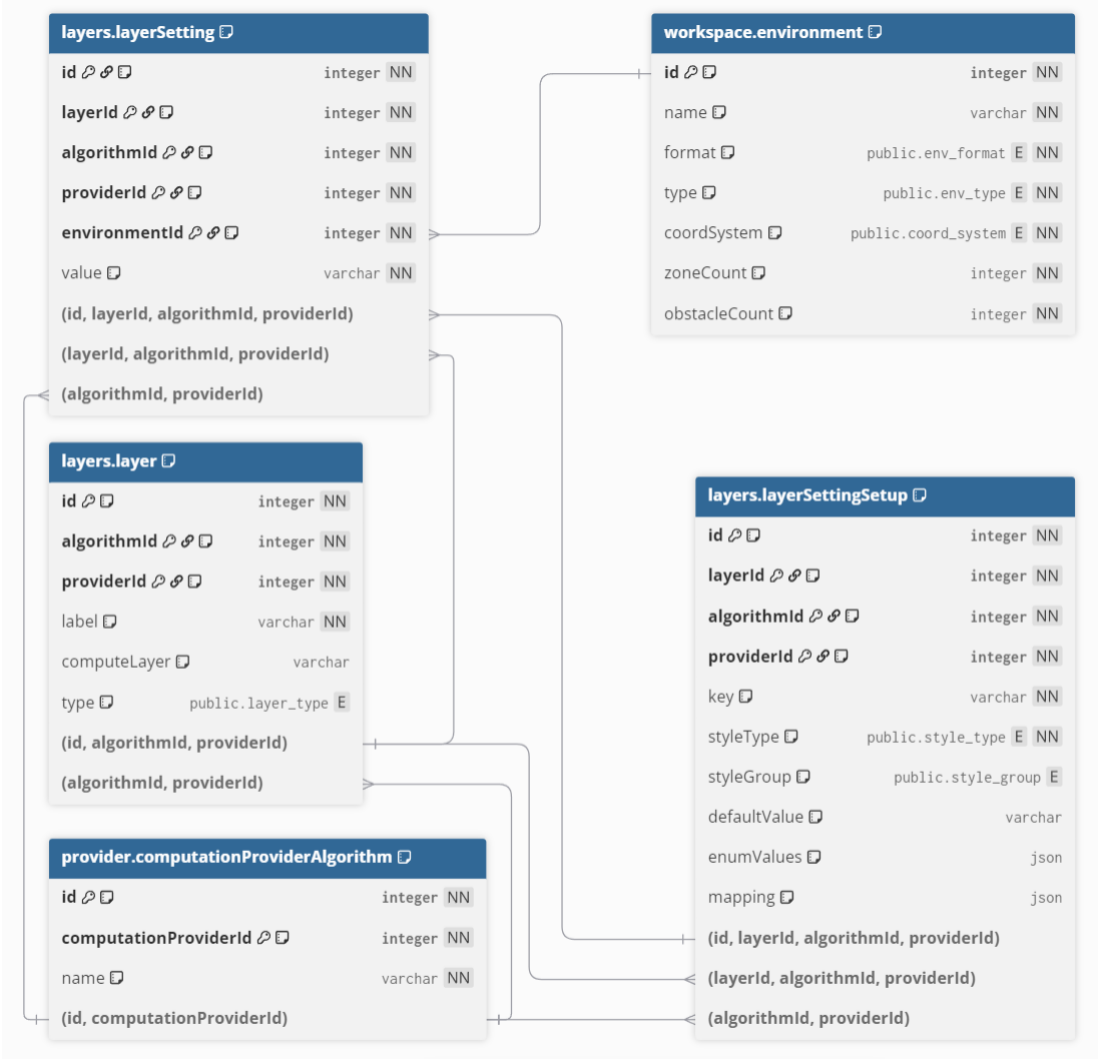
16 pav. *ZPKP vykdymo fizinė duomenų bazės schema*



17 pav. *Algoritmo parametrų fizinė duomenų bazės schema*

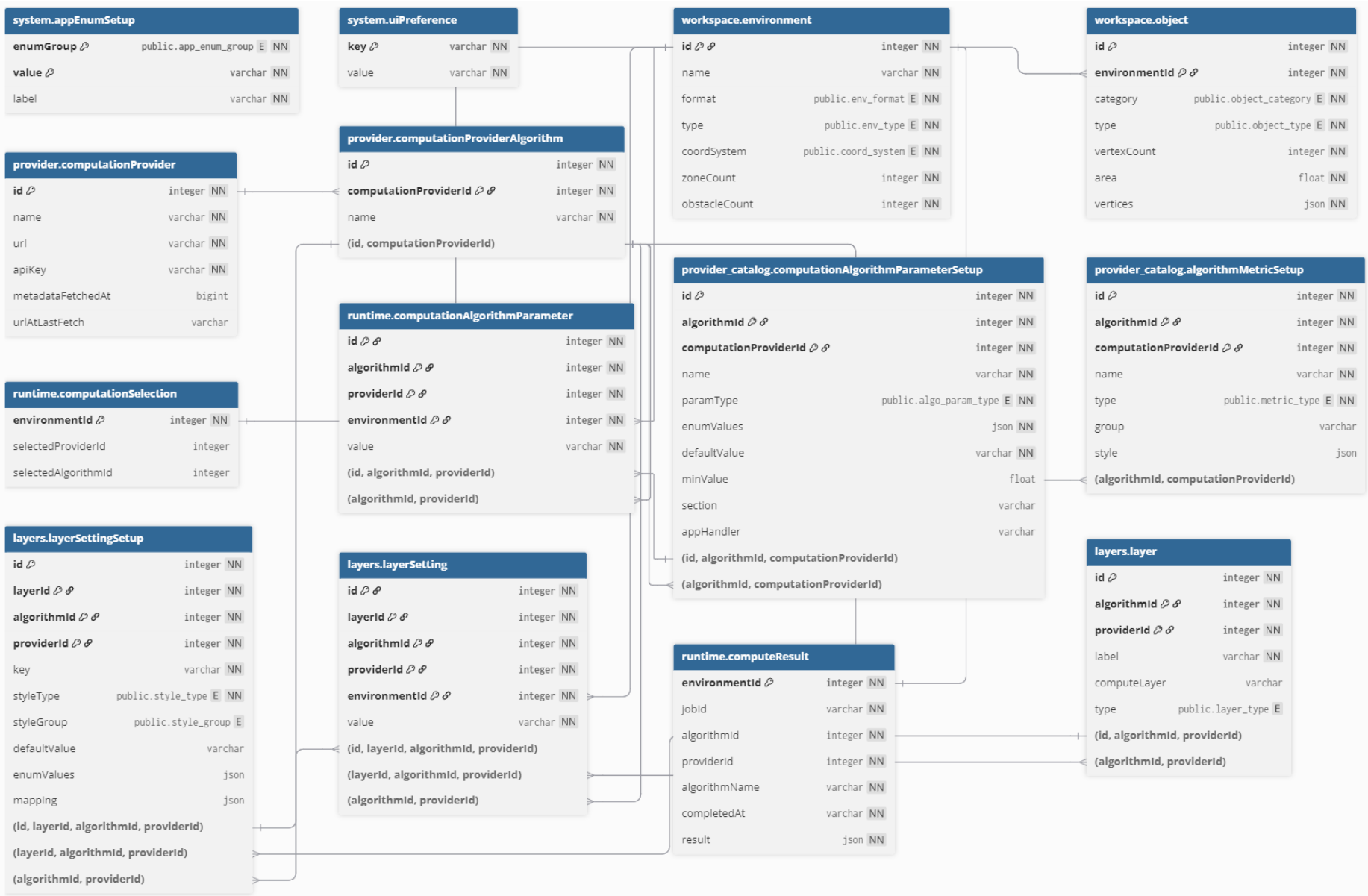
Skaičiavimo rezultatai atvaizduojami sluoksnių principu. Siunčiant palaikomų algoritmų užklausą skaičiavimo serveriui, atsakyme taip pat gaunami metaduomenys apie rezultatų atvaizdavimo sluoksnius, kurie gali būti 2 tipų: maršruto atvaizdavimo arba tarpinių skaičiavimų. Sluoksniai, „layer“ lentelė, priklauso skaičiavimo algoritmui, „computationProviderAlgorithm“ lentelė, ir kiekvienas

sluoksnis turi savo atvaizdavimo parametrus, „layerSetting“ lentelė, kurie yra individualūs pagal pasirinkta aplinką, „environment“ lentelė. Kiekviena aplinka turi atskiras sluoksnių parametrų reikšmes, nors ir naudojamas tas pats algoritmas. Sluoksnių parametrų metaduomenys saugomi „layerSettingSetup“ lentelėje (žr. 18 pav.).



18 pav. *Atvaizdavimo sluoksnių fizinė duomenų bazės schema*

Pilna fizinė duomenų bazės schema pateikiama 19 pav., internetinė duomenų bazės schemos peržiūra pasiekiama [16].

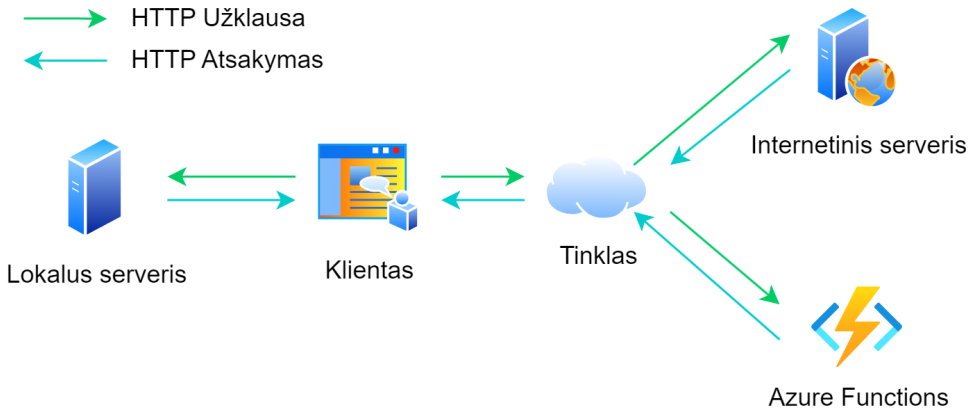


19 pav. *Internetinės naudotojo sąsajos fizinė duomenų bazės schema*

3.12. Internetinės naudotojo sąsajos ir algoritmo sujungimas

Siekiama, kad prie internetinės sąsajos būtų galima prijungti algoritmus su kuo platesne realizavimo įvairove, nemodifikuojant internetinės sąsajos programinio kodo. Šiai sąsajai įgyvendinti panaudota REST API sukuriant komunikacijos kanalą tarp įrankio ir algoritmo naudojant HTTP protokolą. Naudojant REST API sukuriamas informacijos kanalas ne tik tarp viename kompiuteryje veikiančių sistemų, bet ir tarp išorinių sistemų, naudojant papildomas saugumo priemones, API raktus.

Internetinė sąsaja yra klientas, kuris inicijuoja ir valdo skaičiavimo procesus siųsdamas užklausas į serverį, t.y. į algoritmo lokalų ar internetinį serverį arba skaičiavimo paslaugas, tokias kaip „Azure Functions". Algoritmo serveris gavęs užklausas iškart siunčia atgal atsakymus su serverio būsena ir skaičiavimų informacija. Internetinė sąsaja turi nuolatos tikrinti skaičiavimo progresą siunčiant pakartotines užklausas, kol skaičiavimai būna įvykdyti. Toks vienos krypties užklausų siuntimas pilnai užtikrina internetinės sąsajos proceso valdymą šeimininko-pavaldinio (angl. master-slave) principu (žr. 20 pav.).



20 pav. *Šeimininko-pavaldinio architektūra naudojant REST API sąsają*

3.13. Sistemos paleidimas ir naudojimas

Sistemą paleisti ir naudoti galima naudojant Bun įrankius, paleidžiant programuotojo serverį (angl. dev server), tačiau tai reikalauja naudotojui susikonfigūruoti savo darbo aplinką, jog sistema veiktų. Kad naudotojo patirtis būtų paprasta ir greita, naudojami Docker konteineriai, kurie automatiškai paruošia izoliuotą aplinką programos vykdymui ir paleidžia programą. Naudojant konteinerius yra įvykdomas iškeltas reikalavimas, kad sistemos paleidimas nereikalautų specifinių žinių.

4. Įgyvendinimo dalis

Šiame skyriuje pateikiamas sukurtas pirminės versijos interneto sąsajos įrankis darbui su ZPKP algoritmais ir algoritmo skaičiavimo serveris. Sistema tenkina funkcionalumo reikalavimus, tokius kaip: aplinkos konfigūravimas (zonos ir kliūčių apibrėžimas), ZPKP inicijavimas (naudojant naudotojo suvestus skaičiavimo serverio duomenis), algoritmo parametrų keitimas, vykdymo informacijos pateikimas, atvaizdavimo kontrolė, aplinkos išsaugojimas, užkrovimas, importavimas, eksportavimas ir vykdymo metrikų peržiūrėjimas.

4.1. Zonos padengimo kelio planavimo simuliacinės aplinkos įrankio realizavimas

Internetinė vartotoją sąsają galima paleisti naudojant „bun run dev“ komandą, kuri grąžina URL adresą, iš kurio pasiekiamas įrankis. Startavęs įrankis atidaro pagrindinį langą (žr. 21 pav.). Pagrindinis langas sudarytas iš 6 komponentų:

1. Aplikacijos antraštė — komponentas, kuriame atvaizduojamas aplinkos pavadinimas (kurį galima keisti ant jo paspaudus) ir meniu juosta, iš kurios galima įvykdyti didžiąją dalį programos funkcijų.

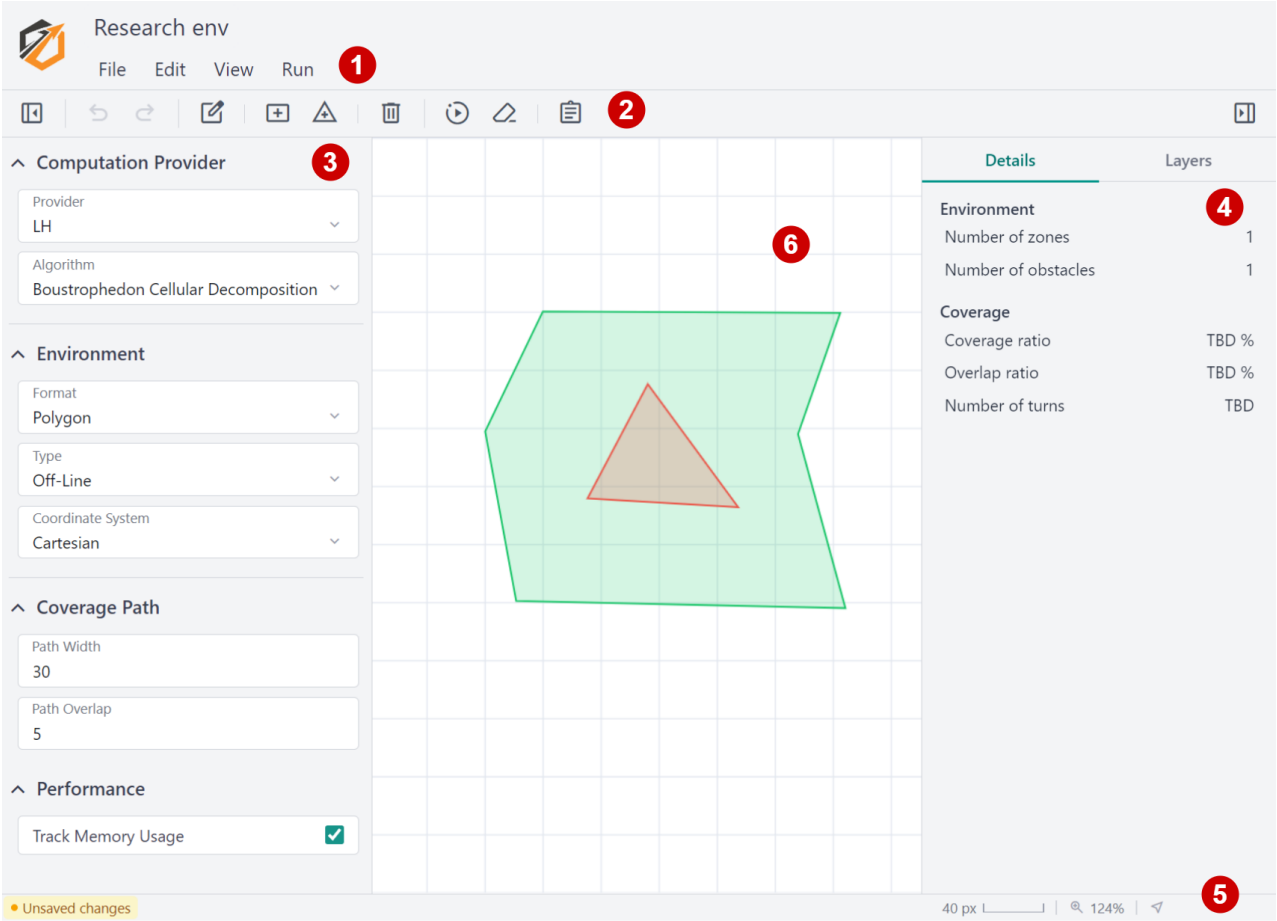
2. Įrankių juosta — funkcijos valdančios aplinkos modifikavimą, skaičiavimo inicijavimą ir šoninių skydelių matomumą.

3. Valdymo skydelis — skaičiavimo serverio, algoritmo pasirinkimas ir algoritmo parametrų nustatymas (parametrai užkraunami dinamiškai pagal skaičiavimo serverio gautus metaduomenis).

4. Informacinis, atvaizdavimo skydelis — pateikia aplinkos ir padengimo metrikas, valdomi atvaizdavimo sluoksniai.

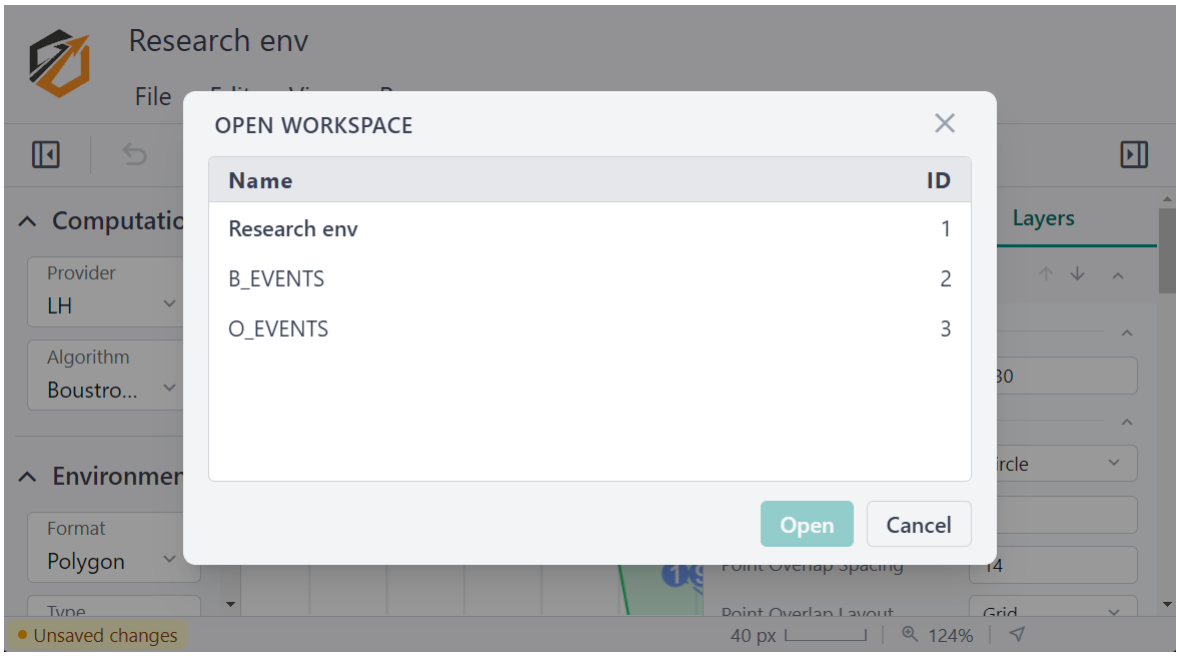
5. Būsenos juosta — pateikia sistemos būsenos informaciją tokią kaip duomenų išsaugojimo, mąstelio, žymeklio pozicijos.

6. Drobės langas — komponentas, kuriame vyksta aplinkos modifikavimas, atvaizduojami gauti zonos padengimo rezultatai.

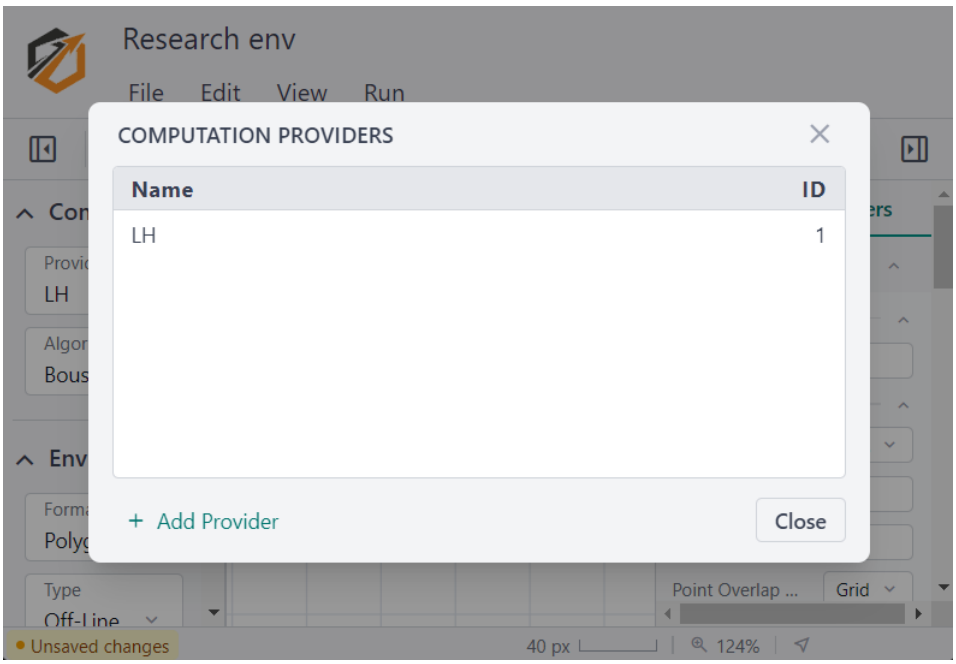


21 pav. *Pagrindinis įrankio langas*

Įrankis sąrašų duomenis pateikia modaliniuose languose (žr. 22 pav.), kurių komponentus galima atidaryti ar trinti. Gali būti naudojamos papildomos funkcijos, tokios kaip naujo įrašo sukūrimas (žr. 23 pav.).

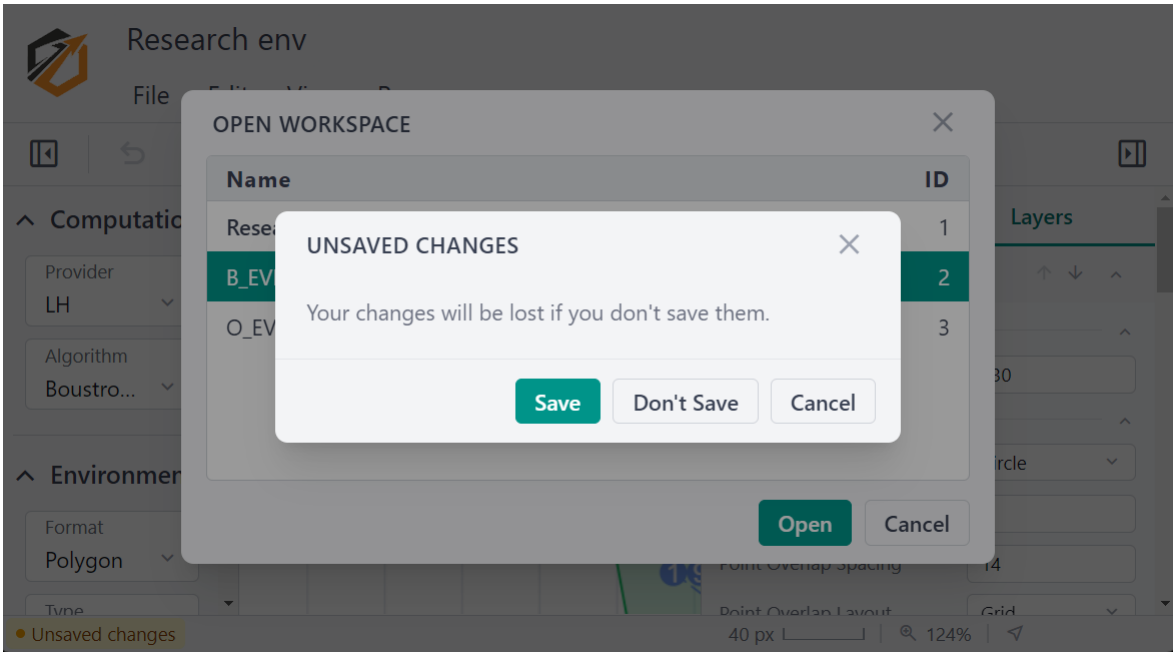


22 pav. *Išsaugotų aplinkų sąrašo langas*

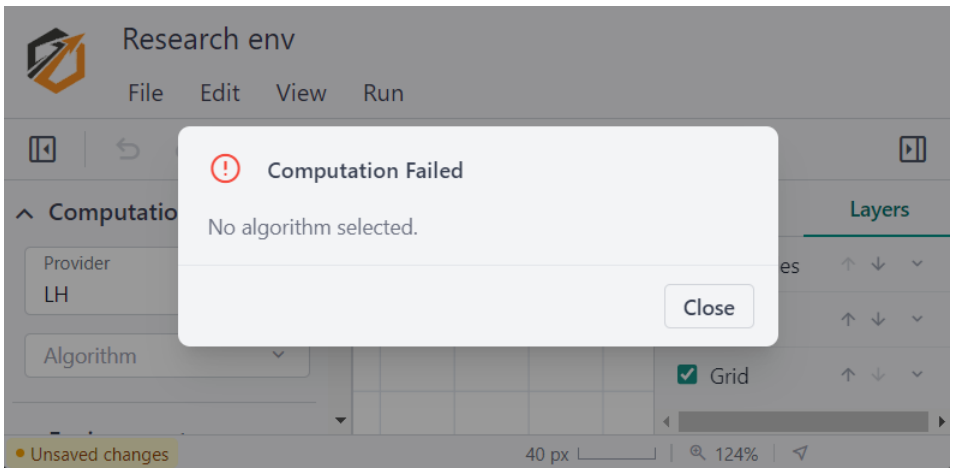


23 pav. *Skaičiavimo serverių sąrašo langas*

Įrankis prieš atlikdamas kažkokius veiksmus, pirmiausiai patikrina, kokia yra sistemos būsena ir jeigu reikalinga atlikti papildomus veiksmus, sistema pateikia patvirtinimo dialogo langus (žr. 24 pav.) Jei atliekamas veiksmas nepavyksta, tuomet parodomas klaidos pranešimo langas (žr. 25 pav.).



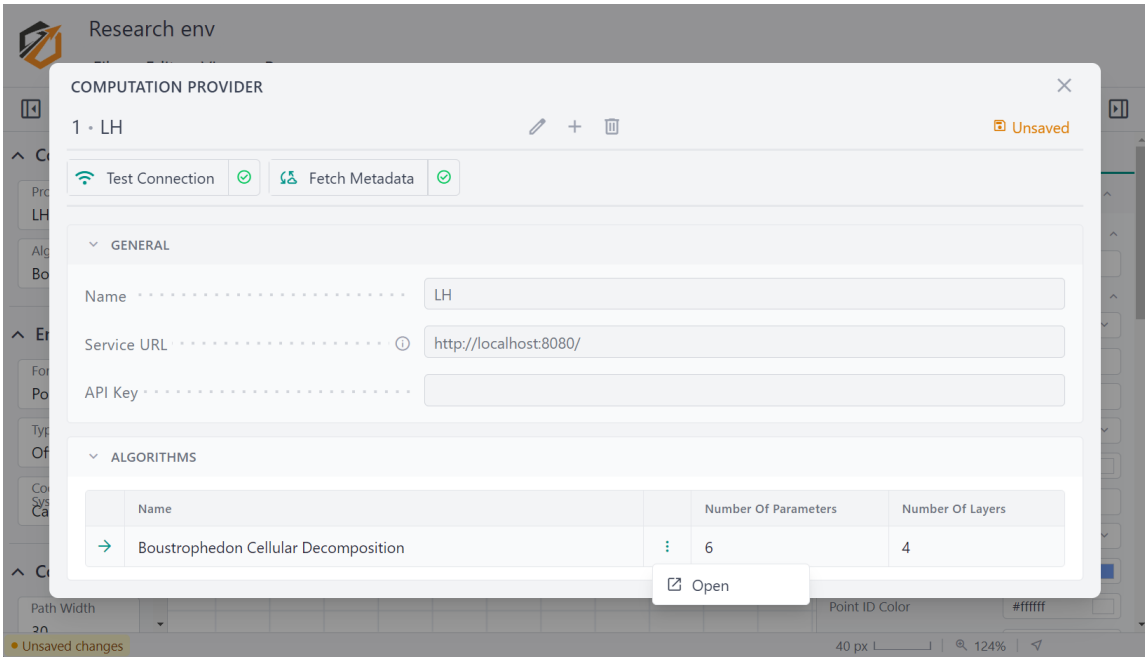
24 pav. *Patvirtinimo dialogo langas*



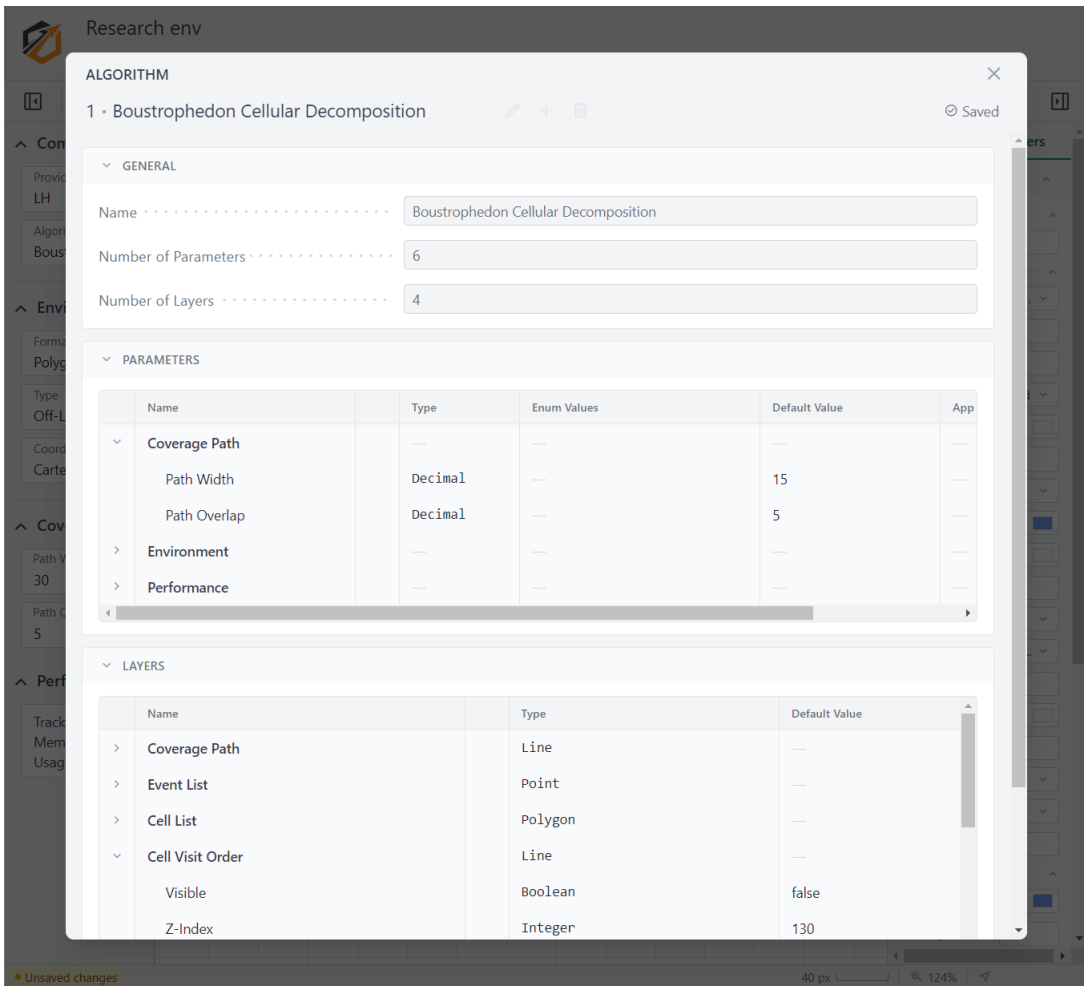
25 pav. *Klaidos pranešimo langas*

Detalesnė informacija pateikiama kortelės languose (žr. 26, 27 pav.). Kortelės gali atvaizduoti laukų informaciją arba integruotus sąrašus. Skaičiavimo serverio kortelėje laukuose pateikiama pavadinimo ir „URL“ informacija, o algoritmai pateikiami integruotame sąraše, per kurį galima pasiekti detalesnę algoritmo informaciją. Algoritmo kortelėje atvaizduojama algoritmo parametrų ir atvaizdavimo sluoksnių metaduomenys.

Skaičiavimo serverio metaduomenys atnaujinami kortelėje (žr. 26 pav.) paspaudus „Fetch Metadata“ mygtuką, kuris išsiunčia užklausą skaičiavimo serveriui ir, jeigu atsakymas gautas sėkmingai, duomenys yra atnaujinami arba parodomas klaidos pranešimas.

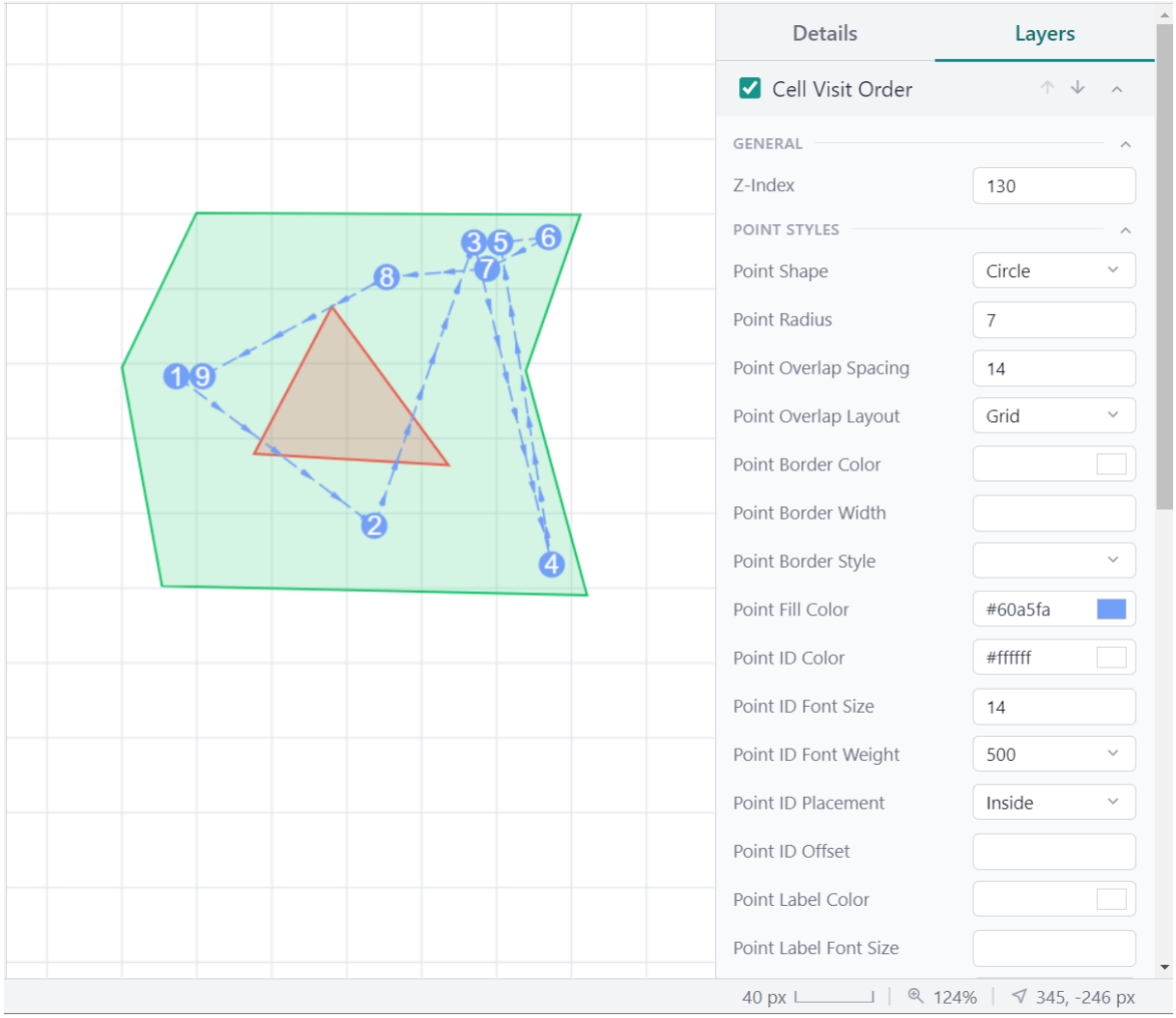


26 pav. *Skaičiavimo serverio kortelės langas*



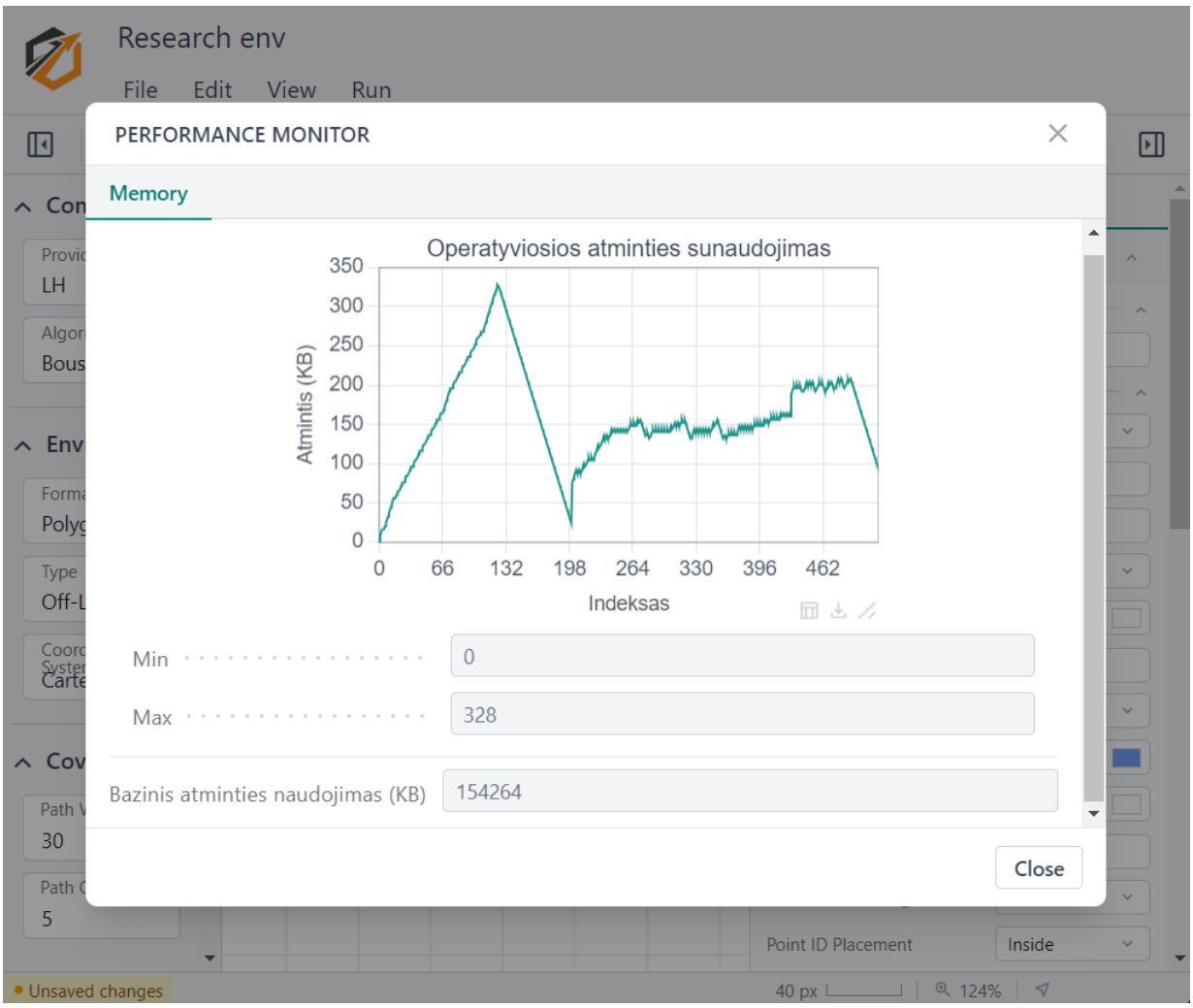
27 pav. *Algoritmo kortelės langas*

ZPKP rezultatų atvaizdavimą galima redaguoti naudojantis informaciniame, atvaizdavimo skydelyje esančiu sluoksnių skirtuku. Skirtuke pateikiami pasirinkto algoritmo sluoksniai. Šių sluoksnių parametrai gali būti keičiami kiekvienai aplinkai atskirai. Tai leidžia nemodifikuojant skaičiavimo serverio metaduomenų keisti atvaizdavimą ir pritaikyti pagal naudojamą aplinką.



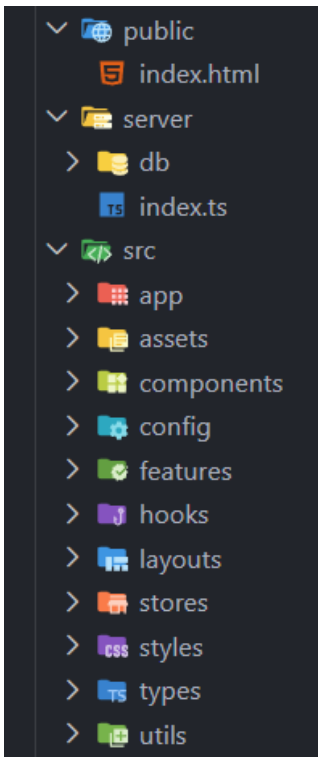
28 pav. *Sluoksnių valdymo skydelis*

Internetinės sąsajos įrankis gali atvaizduoti metrikų duomenis gautas iš skaičiavimo serverio. Tai gali būti pavienės reikšmės arba laiko eilutės, kurios atvaizduojamos grafikais, kurie gali būti eksportuojami „PNG“ arba „CSV“ formatais. Įrankis taip pat prideda „Min“ ir „Max“ laukus kiekvienam grafikui.

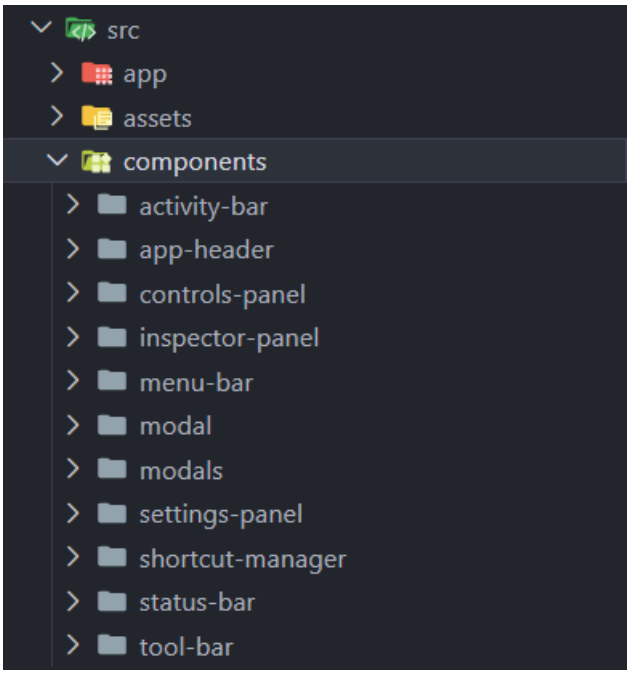


29 pav. *Atlikto skaičiavimo metrikų kortelės langas*

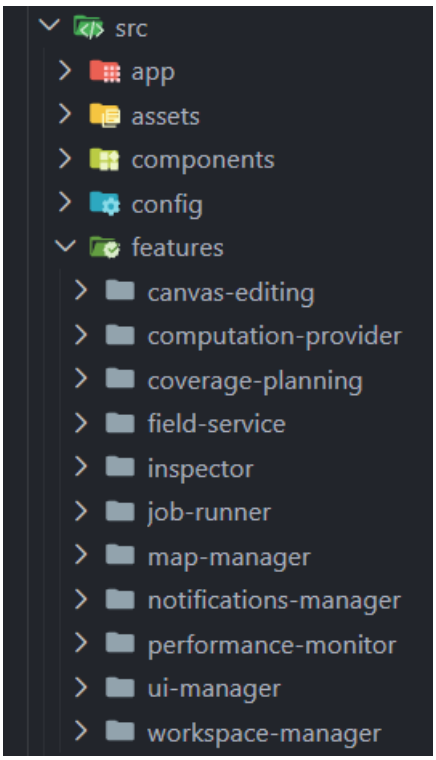
Programinis internetinės svetainės kodas buvo kuriamas struktūrizuotai (žr. 30 pav.), atskiriant skirtingas grupes ir mažinant susietumą nenaudojant globalių komponentų. Bendrai prieinami komponentai yra pagrindas, kuriant specializuotus komponentus (žr. 31 pav.) ir savyje neatlieka jokios procesinės logikos, jie sukurti kaip šablonai, kurie būna užpildomi pagal funkcionalumo poreikį. Procesinė logika ir specializuotų komponentų kūrimas atliekamas atskirose grupėse, kurios tarpusavyje negali sąveikauti (žr. 31 pav.). Kiekviena funkcionalumo grupė turi duomenų valdymo, apdorojimo, atvaizdavimo komponentų įgyvendinimo komponentus (žr. 33 pav.).



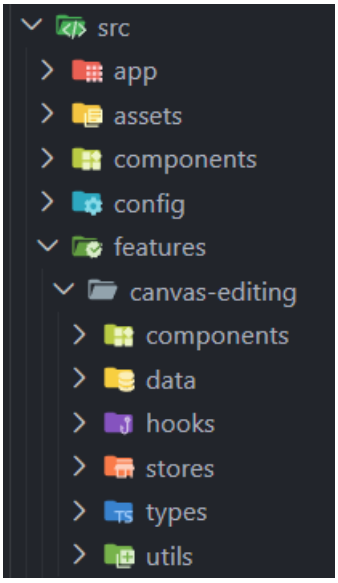
30 pav. *Internetinės svetainės kodo struktūra*



31 pav. *Internetinės svetainės bendri komponentai („activity-bar“, „app-header“ ir t.t)*



32 pav. *Internetinės svetainės įgyvendintos funkcijų grupės („canvas-editing“, „computation-provider“ ir t.t)*

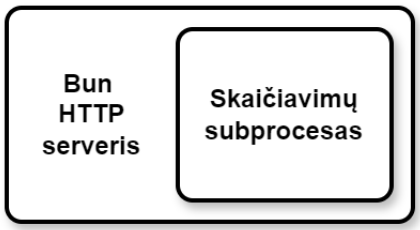


33 pav. *Internetinės svetainės įgyvendintos funkcijos grupės sudėtis („canvas-editing/components“, „canvas-editing/data“ ir t.t)*

4.2. Zonos padengimo kelio planavimo algoritmo realizavimas

Algoritmo skaičiavimo serveris (žr. 34 pav.) veikia „Bun HTTP“ pagrindu. Internetinis serveris priima bei apdoroja užklausas ir siunčia atgal atsakymus. Kai reikia atlikti BCD skaičiavimus, sukuriamas papildomas izoliuotas subprocesas, kuriame atliekami skaičiavimai. Toks dizainas atskiria inter-

netinio serverio ir skaičiavimų gijas, kas leidžia siųsti užklausas į internetinį serverį skaičiavimų metu. Tai leidžia nutraukti skaičiavimus ar sekti skaičiavimų būseną.



34 pav. *Algoritmo skaičiavimų serveris*

Algoritmo skaičiavimo serveris priima 4 tipų „GET“ užklausas:

- „/“ — internetinio serverio informacija, galimos užklausos (žr. 35 pav.).
- „/health“ — serverio būsenos informacija (žr. 36 pav.).
- „/metadata“ — serverio algoritmų metaduomenys (žr. 37 pav.).
- „/compute/<jobId>“ — skaičiavimo rezultatų duomenys (žr. 38 pav.).

Ir vieną „POST“ užklausą — „/compute“. Šioje užklausoje reikia pateikti aplinkos duomenis ir algoritmo parametrus (žr. 39 pav.), serveris iškart siunčia atsakymą (žr. 40 pav.) su skaičiavimo proceso identifikaciniu numeriu, kurį naudojant „GET“ užklausoje „/compute/<jobId>“, atgal gaunami skaičiavimo rezultatai.

```
1  {
2      "service": "blade-provider",
3      "endpoints": {
4          "health": "/health",
5          "metadata": "/metadata",
6          "compute": "/compute",
7          "computeStatus": "/compute/:jobId"
8      },
9      "algorithms": [
10         {
11             "name": "Boustrophedon Cellular Decomposition"
12         }
13     ]
14 }
```

35 pav. *„GET“ užklausa „/“*

```
1  {
2      "status": "ok",
3      "service": "blade-provider",
4      "timestamp": "2026-04-19T22:46:41.398Z"
5  }
```

36 pav. *„GET“ užklausa „/health“*

```
1   {
2       "algorithms": [
3           {
4               "id": 1,
5               "name": "Boustrophedon Cellular Decomposition",
6 >             "parameters": [···
63              ],
64 >            "metrics": [···
81              ],
82 >            "layers": [···
731             ]
732         }
733     ]
734 }
```


37 pav. *„GET“ užklausa „/metadata“*


```
1   {
2       "jobId": "dfa8a3ed-95a7-489b-89a2-f7a42b11add5",
3       "status": "completed",
4       "algorithmName": "Boustrophedon Cellular Decomposition",
5       "createdAt": "2026-04-19T22:48:58.551Z",
6       "startedAt": "2026-04-19T22:48:58.559Z",
7       "result": {
8           "coveragePathPlan": {
9               "segments": [
10                  {
11                      "id": 0,
12                      "type": "coverage",
13 >                    "path": [···
77                      ]
78                  }
79              ]
80          },
81          "debug": {
82              "layers": [
83                  {
84                      "id": 10,
85                      "source": "eventList",
86 >                    "list": [···
111                     ]
112                 },
113                 {
114                     "id": 11,
115                     "source": "cellList",
116 >                   "list": [···
138                     ]
139                 },
140                 {
141                     "id": 12,
142                     "source": "cellVisitOrder",
143 >                   "list": [···
160                     ]
161                 }
162             ]
163         },
164         "performance": {
165 >           "metrics": [···
327             ]
328         }
329     },
330     "completedAt": "2026-04-19T22:48:58.685Z"
331 }
```


38 pav. *„GET“ užklausa „/compute/<jobId>“*

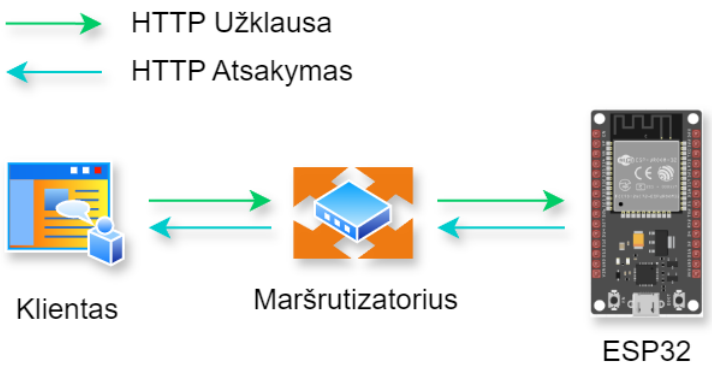
```
{
  "algorithmId": 1,
  "requestId": "optional-client-trace-id",
  "environment": {
    "zones": [
      [
        { "x": 0, "y": 0 },
        { "x": 100, "y": 0 },
        { "x": 100, "y": 100 },
        { "x": 0, "y": 100 }
      ]
    ],
    "obstacles": [
      [
        { "x": 20, "y": 20 },
        { "x": 40, "y": 20 },
        { "x": 40, "y": 40 },
        { "x": 20, "y": 40 }
      ]
    ]
  },
  "parameters": {
    "<parameter name>": "<value>",
    "...": "..."
  }
}
```

39 pav. *„POST“ užklausa „/compute“*

```
{
  "jobId": "550e8400-e29b-41d4-a716-446655440000",
  "status": "queued",
  "createdAt": "2026-03-20T12:00:00.000Z",
  "pollUrl": "http://localhost:8080/compute/550e8400-e29b-41d4-a716-446655440000"
}
```

40 pav. *„POST“ užklausos „/compute“ atsakymas*

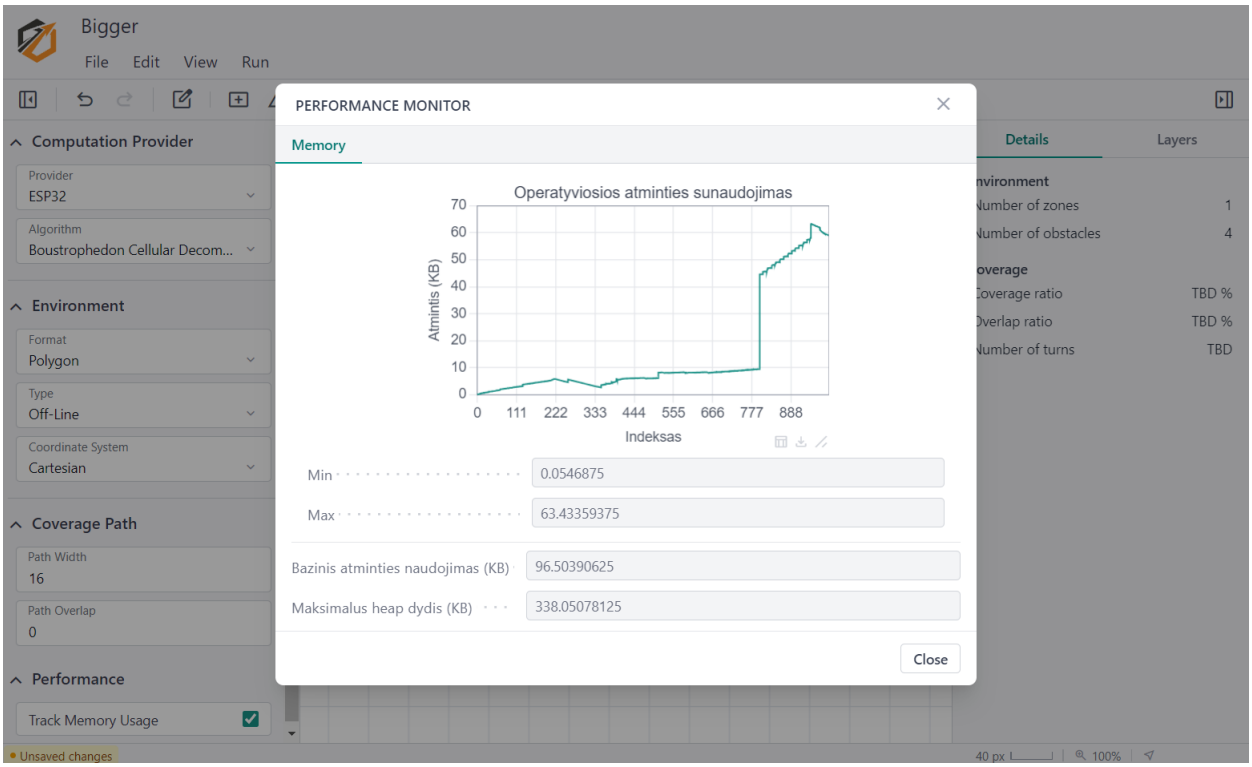
Taip pat algoritmo skaičiavimo serveris buvo įgyvendintas „ESP32“ mikrovaldiklyje (žr. 42, 43 pav.) Komunikacija tarp internetinės naudotojo sąsajos ir mikrovaldiklio buvo atliekama per „Wi-Fi“ tinklą (žr. 41 pav.). Bakalaurinio darbo metu bus siekiama nustatyti, kokias skaičiavimo apimtis būtų galima pasiekti su šiuo paprastu mikrovaldikliu. Tai padėtų nuspręsti, kokią įrangą naudoti robotuose, pagal planuojamų užduočių sudėtingumą.



41 pav. *Komunikacija tarp internetinės naudotojo sąsajos ir „ESP32“*

```
I (3294) wifi: Got IP: 192.168.0.46
I (3294) wifi: Provider base URL: http://192.168.0.46:8080
I (6954) http_server: OPTIONS /compute
I (7014) http_server: POST /compute
I (7544) provider: Job 74f35690-9c17-4591-b140-a3024b472f8a -> completed
I (7964) http_server: GET /compute/74f35690-9c17-4591-b140-a3024b472f8a
```

42 pav. *„ESP32“ veiklos žurnalas*



43 pav. *„ESP32“ algoritmo vykdymo metrikos*

4.3. Zonos padengimo kelio planavimo algoritmo testavimas

Rengiant šią dokumentaciją, BCD algoritmo veikimas buvo ištestuotas rankiniu būdu. Visos algoritmo veikimo iliustracijos (žr. 12 pav.) buvo sukurtos naudojant šią sistemą. Parametrų ir sluoksnių atvaizdavimo funkcionalumas veikia pagal reikalavimus. Automatinis testavimas reikalautų papildomo funkcionalumo, kuris atsitiktinai galėtų kurti aplinkas ir tikrinti ar sugeneruotas kelias nepažeidžia zonos ar kliūčių ribų.

4.4. Skaičiavimo pajėgumų testavimas

Sukurta sistema leidžia skaičiavimo resursų metrikas gauti ir atvaizduoti (žr. 29 pav.) bei atlikti jų analizę. Tolimesni tyrimai ir analizė bus atliekama bakalaurinio darbo metu. Bet prieš tai internetinės svetainės įrankis turės būti praplėstas, pridedant sudėtingesnių aplinkų generavimo funkcionalumą, kad aplinką sudarančių zonos ir kliūčių viršūnių nereiktų visų sudėlioti ranka. Tai leistų testuoti naudojant aplinkas su tūkstančiais ar milijonais taškų.

5. Rezultatai ir apibendrinimas

Profesinės praktikos metu buvo susipažinta su internetinių svetainių kūrimu, skirtingais įrankių rinkiniais, naudojamomis programavimo kalbomis, naudotojo sąsajos karkasais bei duomenų išsaugojimo būdais. Atlikta ZPKP algoritmų analizė parodė, koks technologijų rinkinys reikalingas internetinės sąsajos ir skaičiavimo serverio sistemos kūrimui.

Mokomosios praktikos metu suprojektuotos sistemos pirminė versija buvo įgyvendinta šios praktikos metu, įgyvendinti 22 funkciniai reikalavimai, sukurta 14 duomenų bazės lentelių. Sukurta sistema leidžia tyrėjams interaktyviai kurti aplinkas su zonomis ir kliūtimis, keisti algoritmų parametrus, vykdyti pavienes skaičiavimo užklausas, analizuoti rezultatus ir keisti atvaizdavimo nustatymus.

Įgyvendintas Bustrofedono dekompozicijos algoritmas, kuris veikia kaip atskira sistema, vykdant komunikaciją su internetine vartotojo sąsaja per „HTTP REST API“ sąsają. Algoritmo veikimas buvo ištestuotas rankiniu būdu sukurtoje internetinėje naudotojo sąsajoje, algoritmas veikia kaip numatyta, sėkmingai generuoja zonos padengimo kelio maršrutus zonoms su statinėmis kliūtimis.

Iš viso sukurtos 3 atskiros sistemos: 1 sistema internetinės naudotojo sąsajos ir 2 sistemos algoritmo skaičiavimo serverio, viena veikianti staliniame kompiuteryje (ar bet kokioje kitoje aplinkoje, kuri palaiko „Docker“ konteinerius), kita veikianti „EPS32“ mikrokontroleryje. Sukurtų sistemų kodo apimtis (angl. Count Lines of Code, CLOC):

- Internetinė vartotojo sąsaja — 15678 CLOC,
- Algoritmo skaičiavimo serveris „Docker“ konteineryje — 4317 CLOC,
- Algoritmo skaičiavimo serveris „ESP32“ mikrokontroleryje — 4122 CLOC.

Praktikos metu įgyta patirtis internetinių svetainių programavimo, paskirstomųjų sistemų kūrimo, skaičiavimo algoritmų realizavimo srityse. Plėtotos analitinės kompetencijos, atliekant detalesnę algoritmo analizę ir identifikuojant tinkamą technologijų rinkinio pasirinkimą. ZPKP tema toliau bus atliekamas bakalaurinis darbas, siekiant realizuoti kitus ZPKP algoritmus ir atlikti jų palyginamąja analizę.

Literatūra

[1] J. Bogner, M. Merkel. „To type or not to type? a systematic comparison of the software quality of javascript and typescript applications on github“. Iš: *Proceedings of the 19th international conference on mining software repositories*. 2022, puslapiai 658–669.

[2] Z. L. Cao, Y. Huang, E. L. Hall. „Region filling operations with random obstacle avoidance for mobile robotics“. Iš: *Journal of Robotic Systems* 5.2 (1988), puslapiai 87–102.

[3] H. Choset. „Coverage for robotics—a survey of recent results“. Iš: *Annals of Mathematics and Artificial Intelligence* 31.1 (2001), puslapiai 113–126. URL: `https://link.springer.com/content/pdf/10.1023/A:1016639210559.pdf`.

[4] H. Choset, P. Pignon. „Coverage path planning: The boustrophedon cellular decomposition“. Iš: *Field and Service Robotics*. Springer, 1998, puslapiai 203–209. URL: `https://www.ri.cmu.edu/pub_files/pub4/choset_howie_1997_3/choset_howie_1997_3.pdf`.

[5] J. Colegrave, A. Branch. „A Case Study of Autonomous Household Vacuum Cleaner“. Iš: *AIAA/NASA CIRFFSS*. 1994.

[6] E. Galceran, M. Carreras. „A survey on coverage path planning for robotics“. Iš: *Robotics and Autonomous Systems* 61.12 (2013), puslapiai 1258–1276. URL: `https://doi.org/10.1016/j.robot.2013.09.004`.

[7] M. Höffmann, S. Patel, C. Büskens. „Optimal guidance track generation for precision agriculture: A review of coverage path planning techniques“. Iš: *Journal of Field Robotics* 41.3 (2024), puslapiai 823–844. URL: `https://doi.org/10.1002/rob.22286`.

[8] M. S. Islam. „JavaScript alternative (TypeScript) and its effectiveness in web development“. Iš: (2023).

[9] K. Kalman, M. Miłosz. „Comperative analasys of JavaScript runtime environments“. Iš: *Journal of Computer Sciences Institute* 36 (2025), puslapiai 296–302.

[10] Konva. *Getting started with React and Canvas via Konva*. 2026. URL: `https://konvajs.org/docs/react/index.html` (žiūrėta 2026-04-07).

[11] Y. Li, H. Chen, M. J. Er, X. Wang. „Coverage path planning for UAVs based on enhanced exact cellular decomposition method“. Iš: *Mechatronics* 21.5 (2011), puslapiai 876–885. URL: `https://doi.org/10.1016/j.mechatronics.2010.10.009`.

[12] Mdn. *Client-side storage*. 2026. URL: `https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Client-side_APIs/Client-side_storage` (žiūrėta 2026-04-07).

[13] G. Mier. *Demo image*. URL: `https://github.com/Fields2Cover/Fields2Cover/blob/main/docs/figures/demo_image.png` (žiūrėta 2022).

[14] G. Mier, J. Valente, S. de Bruin. „Fields2Cover: An Open-Source Coverage Path Planning Library for Unmanned Agricultural Vehicles“. Iš: *IEEE Robotics and Automation Letters* 8.4 (2023), puslapiai 2166–2172. https://doi.org/10.1109/LRA.2023.3248439.

[15] J. Remeikis. *Zonos padengimo kelio planavimo simuliacinės įrangos internetinės sąsajos projektavimas, kūrimas ir diegimas*. 2026. URL: https://github.com/jRmx0/mokomoji-praktika/blob/main/Justas_Remeikis_Mokomoji_Praktika.pdf (žiūrėta 2026-04-14).

[16] J. Remeikis. *ZPKP - Internetinės naudotojo sąsajos fizinė duomenų bazės schema*. 2026. URL: https://dbdiagram.io/d/ZPKP-Internetines-naudotojo-sasajos-fizine-duomenu-bazes-schema-69e3d32b0aa78f6bc10832d3 (žiūrėta 2026-04-19).

[17] N. Rottmann, R. Denz, R. Bruder, E. Rueckert. „A probabilistic approach for complete coverage path planning with low-cost systems“. Iš: *Proc. 2021 European Conf. Mobile Robots (ECMR)*. 2021, puslapiai 1–8. URL: https://ieeexplore.ieee.org/document/9568847.

[18] N. Shah, U. Dey, K. Nishimiya. *End-to-End Framework for Robot Lawnmower Coverage Path Planning using Cellular Decomposition*. arXiv preprint arXiv:2506.06028. 2025. URL: https://arxiv.org/pdf/2506.06028 (žiūrėta 2026-01-04).

[19] C. S. Tan, R. Mohd-Mokhtar, M. R. Arshad. „A comprehensive review of coverage path planning in robotics using classical and heuristic algorithms“. Iš: *IEEE Access* 9 (2021), puslapiai 119310–119342. URL: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9523743.

[20] R. Vyas. „Comparative analysis on front-end frameworks for web applications“. Iš: *International Journal for Research in Applied Science and Engineering Technology* 10.7 (2022), puslapiai 298–307.

1 priedas. Sukurtos sistemos programinis kodas

Šiame priede pateiktos nuorodos į sistemos programinio kodo repozitorijas.

Internetinės naudotojo sąsajos programinis kodas:

`https://github.com/jRmx0/blade-terminal`

Algoritmo skaičiavimo serverio programinis kodas:

`https://github.com/jRmx0/blade-provider`

Algoritmo skaičiavimo ESP32 serverio programinis kodas:

`https://github.com/jRmx0/blade-provider-esp32`

2 priedas. Dirbtinio intelekto įrankių naudojimo deklaracija

Buvo naudojami 2 dirbtinio intelekto įrankiai skirtingoms užduotims:

- **VS Code Copilot (Claude Sonnet 4.6)**: kodo tikrinimui, klaidų taisymui, idėjų generavimui, pavyzdžių generavimui, grafikų, vartotojo sąsajos generavimui, gerai žinomų algoritmų (BFS, DFS) generavimui, duomenų apdorojimo funkcionalumo generavimui.
- **Kablelis.lt**: žodžių rašybos, gramatikos ir skyrybos klaidų taisymui, sakinių perfrazavimui.